

DTScribe: Generating Models for Digital Twin Reporting using Local Large Language Models

Kérian Fiter^{1*}, Angelica Portocarrero² and Bentley Oakes¹

¹Département de Génie Informatique et Génie Logiciel, Polytechnique Montréal, 2500, chemin de Polytechnique, Montréal, H3T 1J4, Québec, Canada.

²Facultad de Ingeniería y Ciencias Básicas, Universidad Autónoma de Occidente, Carrera 25 #115-85, Santiago de Cali, 760030, Colombia.

*Corresponding author(s). E-mail(s): kerian.fiter@polymtl.ca;

Contributing authors: maria.a.portocarrero@uao.edu.co; bentley.oakes@polymtl.ca;

Abstract

The development of a Digital Twin (DT) brings together many stakeholders and technical components. We have previously investigated the reporting of the conceptual architecture for these DTs as an artifact for discussion. However, we find that the activity of modeling the DT for visualization and consensus-building can be quite tedious and time-consuming.

In this article, we explore automation of model generation using LLMs, targeting the model required for DT reporting. Our approach is designed to ingest DT documents, and output both the conceptual architecture visualization and a reporting page for that DT. We first extract from the input the key details aligning with our 21-characteristic DT Description Framework described in previous work. Then, we generate a description model in the Ontology Modeling Language (OML), representing the DT as a constellation of models, data, enablers, and services. We utilize refinement loops with LLMs-as-a-judge and ontological reasoning to increase artifact quality as measured by our metrics.

We investigate how architectural and parameter choices affect artifact quality, such as how refinement loops enable local LLMs to produce sufficiently high-quality results comparable to larger models. We also detail insights and lessons learned useful for practitioners, such that they can integrate our techniques to accelerate their DT reporting or for other model generation tasks. These insights include the use of medium-sized local models (around 20B), avoidance of LLM-as-a-judge and clustering for characteristic extraction, and the recommendation of at least two reasoner loops for OML refinement.

Keywords: Digital Twins, ontologies, ontology engineering, Large Language Models, reporting, Ontological Modeling Language, model generation, ontology refinement loop

1 Introduction

Digital Twins (DTs) are digital representations of physical systems, enabling monitoring, insight generation, and control of their physical counterparts through real-time, bidirectional data exchange [1]. Their use spans multiple industries,

from manufacturing to construction, healthcare, city planning, and robotics. Academic research is also increasingly engaging with DTs, recognizing them as a rapidly expanding area of study, although many challenges remain [2]. In particular, DTs are complex software systems often integrated across entire life-cycles, from design and

manufacturing to operation and decommissioning. DTs and their associated physical systems—which we will call the Physical Twin (PT)—also tend to continually evolve [3], adding to the complexity of those systems.

To handle this dynamic and complex nature, stakeholders require continuous and detailed *reporting* of their characteristics and capabilities to understand the structure and behavior of DTs at any time [4]. Reporting on DTs has proven to be a time-consuming and unstructured process, both for experts to reach agreement on common definitions, and for non-experts to easily understand the expert’s view on a DT [5, 6]. Having a common view of the system facilitates interdisciplinary collaboration by improving understanding and communication across teams, and reduces ambiguity of the system’s technicalities.

1.1 Challenges

In our previous work, we introduced DTInsight [7], a tool to assist DT reporting. DTInsight enables explicit reporting through ontology-based representations, interactive reporting via conceptual architecture generation combined with data monitoring and 3D visualization, and continuous reporting by automatically generating reporting pages within a CI/CD pipeline. We proposed modeling DTs using our twenty-one characteristic DT Description Framework (DTDF) [4], formalized as an ontology. A Godot engine-based visualizer then queries this model to produce a conceptual visualization of the DT’s structure, of its data flows, and of selected aspects of its behavior.

However, through our discussions with DT practitioners and researchers, there are still numerous barriers to widespread use of DTInsight and DT reporting in general. More precisely, we aim to address the following challenges:

Challenge 1: Slowness and difficulty of modeling domain knowledge for reporting

Small and Midsize Enterprises (SMEs) and researchers have limited experience and time to devote to domain knowledge modeling—and its formal, logic-based subset called Ontology Engineering (OE)—as it is slow, labor-intensive, and reliant on expert modelers [8–11].

Furthermore, while recent research has focused on Ontology Learning (OL) [12–18] (the automatic discovery of a domain’s *ontological structure*), little work has been done on *instance model generation* [19] (the automatic generation of *ontology instances* conforming to a certain metamodel).

Challenge 2: Privacy and sovereignty concerns

DT reporting capabilities are often limited by data privacy and sovereignty requirements in industrial sectors [20, 21]. DT documentation often contains sensitive or proprietary data that cannot be processed through cloud-based infrastructures. Consequently, there is a need for solutions that operate *locally* under moderate hardware constraints, to align with the typical resources and technical expertise available in SMEs [22].

To clarify our use of the term “*locally*”, we refer to processing the documentation on the premises of the organization. This can include desktop and laptop machines, as well as on-premise servers. Another way to put this is that we focus on a ‘fog-based’ approach, rather than a ‘cloud-based’ approach.

Challenge 3: Closing the gap between static documents and continuous reporting for non-technical personnel

To deliver insights to diverse stakeholders throughout the DT life-cycle, reporting must be accessible to non-technical personnel. This necessitates an architecture capable of generating multi-view representations [23], allowing users with varying expertise to interpret results within agile processes in a TwinOps style [24].

While we partially addressed these requirements through DTInsight and the DTDF, a significant gap remains between static project documentation and the dynamic generation of reports. Specifically, current methods struggle to align reports with evolving DT data and processes [3], particularly when reports are not treated as first-class artifacts within CI/CD pipelines.

1.2 Proposed Approach

We present the *DTScribe* approach to these challenges:

Addressing Challenge 1 : We tackle the slowness and difficulty of modeling by *leveraging LLMs to extract as much of the twenty-one DTDF characteristics as possible from unstructured user-provided documents*. We use those characteristics to then generate an OML description model within an LLM-based ontology repair loop.

Addressing Challenge 2 : We tackle the privacy and sovereignty concerns by targeting *open-source LLMs running on constrained, local infrastructure*. Unlike larger proprietary, cloud-hosted LLM services that rely on extensive context windows to process documents, our approach optimizes performance for smaller, privacy-preserving models. We achieve this through LLM-based evaluation for characteristic extraction and LLM-based ontology repair for OML description quality.

Addressing Challenge 3 : We target non-domain experts in DT reporting by *generating an ontology compatible with the DTInsight tool [7]*. Finally, we tackle CI/CD by *making it easy to integrate into the continuous pipeline* of DTInsight.

Novelty and Impact

Our first novelty is the automation of DT reporting with privacy-respecting and open-source LLMs to facilitate domain knowledge modeling. Specifically, we propose a novel technique for generating OML description instance models that utilizes an OML refinement and repair mechanism. This allows our approach to match the output quality of larger LLMs. Compared to previous research focusing on OWL, OML is also noticeably less verbose. We hypothesize that this aspect may lead to improved success in generating OML as compared to OWL. The generated ontology integrates with DTInsight to provide continuous DT reporting to non-technical personnel. We also define quality metrics for this process, enabling the evaluation of our model generation approach.

The primary impact of this research is to provide a robust, systematic methodology and tooling that enable users to derive an initial DT conceptual model from artifacts such as documentation. This model then serves as the foundation for reporting throughout the DT life-cycle. That is, the user (or other LLM agents) could make smaller edits to this model to keep it aligned with the DT as it evolves.

1.3 Research Questions

This paper answers the following Research Questions (RQs):

RQ1: What is a **technical architecture with local LLMs** to enable the extraction of DT characteristics and the generation of valid OML description models?

RQ2: How can **LLM-generated textual reports be evaluated** in terms of extracting complete and high-quality DTDF characteristics?

RQ3: How do **choices made in our approach improve the quality of the generated artifacts**, such as LLM-based evaluation, ontology-based repair, and LLM selection?

1.4 Contributions and Structure

This paper makes the following contributions:

1. **Automated Reporting Framework:** A novel approach (available on GitHub¹) for automating DT reporting that leverages local, privacy-respecting LLMs to extract domain knowledge from unstructured documents and generate formal OML instance models.
2. **Feedback Architecture:** A technical architecture that employs iterative refinement loops to overcome the limitations of smaller, local models. These loops are 1) LLM-as-a-judge for characteristic refinement and, 2) reasoner-guided repair for ontology generation.
3. **Empirical Evaluation:** An evaluation of the approach across multiple LLMs, analyzing the trade-offs between model size, deployment modality, and the quality of the generated reports and models.
4. **Tool Integration:** The integration of the generated artifacts into the DTInsight tool to bridge the gap between static documentation and continuous DT reporting.

An example of a reporting page generated by our approach for the exemplar paper [25] can be found here².

This article provides actionable insights for researchers and practitioners making architectural

¹<https://github.com/oakeslabmtl/DTScribe>

²<https://oakeslabmtl.github.io/dtdf-dtscribe/>

decisions regarding local LLMs for DT reporting. Specifically, we evaluate the critical trade-offs between computational cost and extraction accuracy, highlighting the surprising pitfalls of document clustering and the efficacy of iterative, reasoner-guided repair. Consequently, we provide concrete recommendations on optimal model size, the use of LLM-as-a-judge mechanisms, and strategies for reliable OML generation.

The remainder of this paper is organized as follows. Section 2 reviews the necessary background. Section 3 details our proposed approach. Section 4 defines the evaluation metrics, while Section 5 describes the experimental setup. Section 6 presents and analyzes the empirical results. Section 7 discusses implications, limitations, and threats to validity. Section 8 situates our contributions within the broader literature. Finally, Section 9 concludes the paper.

2 Background

This section discusses the key concepts and technological foundations underlying DT construction, reporting, and ontology-based modeling. We focus on the emerging role of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) in supporting these processes.

2.1 DT Reporting with DTInsight

DT Engineering (DTE) increasingly relies on Model-Driven Engineering (MDE) [26], defined as “the use of software models to design, analyze, generate, and document software” [27]. Inspired by the use of ontologies for DT construction and architectural definition [28–30], we propose leveraging ontologies for DT reporting using LLMs. This approach aligns DTE with formal ontological representations and the emerging automation capabilities enabled by Foundation Models (FM) and Generative AI [31–36].

This convergence of models, ontologies, and AI [37] contrasts with the current state of DT reporting, which remains under-specified [4], particularly regarding the full DT life cycle. This gap persists despite the increasing demand for continuous reporting, often referred to as TwinOps [24], and emerging standards like ISO 23247 [38], which recognizes reporting as a key *functional entity*.

DT Description Framework (DTDF)

To structure this domain, the DT Description Framework (DTDF) [4] characterizes DTs across twenty-one dimensions. These range from functional aspects like *DT services (C6)*, which defines capabilities such as simulation or monitoring, to lifecycle management via the *Twinning process and DT evolution (C13)*, and governance concerns regarding *Data ownership and privacy (C19)*. The framework facilitates consensus among diverse stakeholders enabling them to define each system aspect in a few targeted sentences and *constellation diagram* [6]. This conceptual structure was subsequently formalized into an ontology, as detailed in the following sections.

DTInsight

Our tool DTInsight [7] addresses the lack of structured, automated reporting by leveraging “the symbiotic relationship between MDE and DevOps” [39]. It integrates ontology-based DT descriptions into a continuous reporting pipeline to generate two primary artifacts: a web-based reporting page³ and a visual *DT constellation* (Figure 1). These artifacts are designed to serve as an initial draft for discussion and iterative modification, enabling stakeholders to grasp the current characteristics and conceptual architecture of the DT. The interactive constellation allows users to explore data flows between components, view underlying scripts, and visualize real-time sensor data through graphs or 3D models.

DTInsight models the DTDF within an ontology (publicly available here⁴) written in the Ontology Modeling Language (OML) [40], which is described later. If the generated OML description conforms to the DTDF vocabulary, the tool automatically generates the *DT constellation* and the reporting page. However, the creation and maintenance of the underlying DTDF ontology currently remain largely manual and effort-intensive, limiting the scalability and broader adoption of ontology-driven DT reporting.

Running Example: Incubator Case Study

Following the DTInsight article [7], we adopt the incubator DT case study [41] as a running example

³<https://oakeslabmtl.github.io/DTDF>

⁴<https://github.com/oakeslabmtl/DTDF>

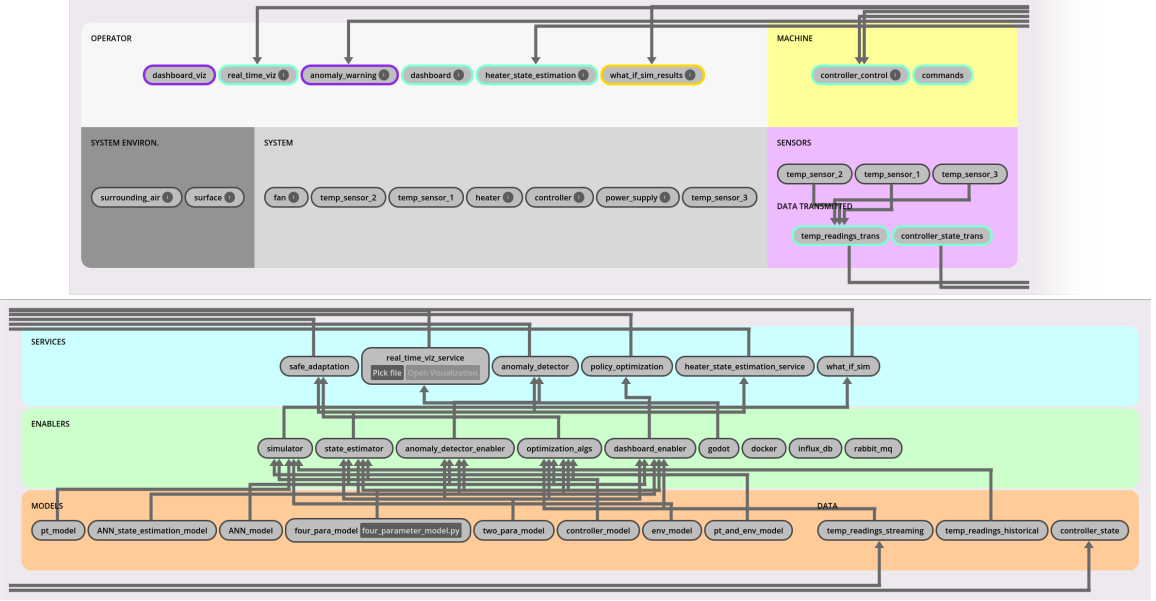


Fig. 1: The incubator [41] DT constellation [4] automatically generated by DTInsight from the (hand-built) OML description. Interactive reporting page available at <https://oakeslabmtl.github.io/DTDF/>.

throughout this work. The objective of this system is to regulate the temperature within an enclosed box by controlling a heater, based on measurements collected from multiple temperature sensors and a corresponding thermal model.

In DTInsight, this case study was selected due to the simplicity of its control objective combined with the complexity of its behavior, which covers multiple domains including electrical, thermal, and mechanical. Figure 1 illustrates the DTInsight constellation representing the incubator DT.

The original publication of the case study spans eighteen pages and covers multiple aspects of the DT’s realization such as hardware components, software architecture, model definition, calibration procedures, and operational goals. Notably, this description is provided without relying on any standardized reporting framework. This lack of structure makes the incubator DT an especially suitable candidate for our study, as it allows us to evaluate the feasibility of automatically deriving a DT reporting model in accordance with the DTDF.

2.2 Ontologies and OML

An ontology is “an explicit specification of a conceptualization” [42]. Here, we write our ontology

in the Ontology Modeling Language (OML) using the openCAESAR platform [40] from NASA’s Jet Propulsion Laboratory (JPL).

OML is increasingly being considered for Model-Based System Engineering (MBSE) [43], especially in the case of DTE [30]. This is because OML can combine the expressiveness of formal ontologies with practical MBSE workflow integration, providing precise semantic definitions and automated consistency checking while enabling both human-readable specifications and machine-processable reasoning.

OML provides a Domain-Specific Language (DSL) for ontology modeling, distinguishing between two complementary layers: an OML *vocabulary*, which defines domain concepts, constraints, and their relations (similar to an M1 metamodel), and an OML *description*, which instantiates these concepts using the vocabulary (similar to a M0 model). A slice of the DTDF OML vocabulary is presented in Listing 1, and its instantiation as an OML description on Listing 5.

Our model generation approach thus touches on *Ontology Learning* (OL), which is concerned with the (semi-)automatic discovery and extraction of ontological structures from raw data. The advent of LLMs has prompted substantial research interest in leveraging them to advance OL [12–18].

```

// C2: Acting Components
concept ActingComponent < ConnectionComponent,
    TimeScaleThing
// C3: Sensing Components
concept SensingComponent < ConnectionComponent,
    TimeScaleThing
// C4: Physical to Virtual Interactions
concept DataTransmitted < ConnectionComponent,
    TimeScaleThing, AutomaticEnabledThing
relation entity ProducedData [
    from SensingComponent
    to DataTransmitted
    forward producedData
    reverse producedFrom
]

```

Listing 1: DTDF OML vocabulary example.

However, our research focuses on a similar yet distinct area: *instance model generation* [44]. While OL typically revolves around the discovery of conceptual structures from data, instance model generation assumes the existence of a metamodel that formally represents the abstract system structure. The goal is then to generate instances that match the structure and constraints of the metamodel. As with OL, there has been interest in using LLMs for instance model generation [19], but to a lesser degree.

In our work, the pre-existing metamodel is the DTDF [4] OML vocabulary, which has been formally represented in DTInsight [7]—with the full vocabulary publicly available online⁵. Here, the *instance model* is the DTDF OML description.

2.3 Large Language Models

Large Language Models (LLMs) are transformer-based neural networks trained on vast text corpora, enabling them to perform a wide range of natural language tasks such as question answering, code generation, translation, and text summarization [45]. However, their knowledge remains limited to the scope of the data available during training, inheriting not only factual information but also the potential biases and misconceptions present in those sources. Moreover, when processing long text sequences, LLMs may omit important contextual details, leading to incomplete or incorrect responses [46].

To address these limitations, several methods have been adopted. *Retrieval Augmented Generation* (RAG) [47] supplements LLMs with external

domain knowledge by retrieving relevant information from a vector database at inference time. In RAG, documents are split into smaller passages (chunks), embedded into vector representations, and stored in a vector database. During inference, a user query is embedded and the k most semantically similar chunks are retrieved and appended to the model’s context.

RAG thus allows the LLM to ground its response on up-to-date, domain-specific information. This method is effective for Information Extraction (IE) tasks [48], where relevant data may be spread across multiple sections of text, and for Aspect-Based Summarization (ABS), which condenses unstructured text into focused summaries on specific topics [49]. Combining LLMs with embedding-based retrieval can reduce hallucinations by grounding the model’s responses in actual, relevant content, improving both extraction accuracy and the quality of summaries [50].

LLMs often face difficulties when a prompt contains multiple rules, multi-step reasoning, and strict structured output requirements. To guide LLMs, the *prompt chaining* technique is commonly employed to decompose a workflow into sequential, narrower prompts. This especially improves output quality for smaller models with a small context window. Similarly, the *Chain-of-Thought* (CoT) prompting technique has been shown to improve output quality [51] by explicitly structuring complex prompts as a step-by-step reasoning process. However, recent research into LLMs has shown that both techniques are becoming outdated as even small LLMs now support large context windows and directly implement reasoning in their architecture [52].

3 Approach

This section details our approach, the objective of which is to ingest user documents and produce a DTDF description model for automatically generating the *DT constellation* figure in DTInsight.

As illustrated in Figure 2, the technical process begins with user-provided DT documents (A), from which an LLM (B) extracts characteristics (C) guided by the DTDF. These characteristics are fed to another LLM for ontology generation (E). To enhance output quality, we integrate two optional refinement loops: an *LLM-as-a-judge* evaluation loop (D) to iteratively

⁵<https://github.com/oakeslabmtl/DTDF>

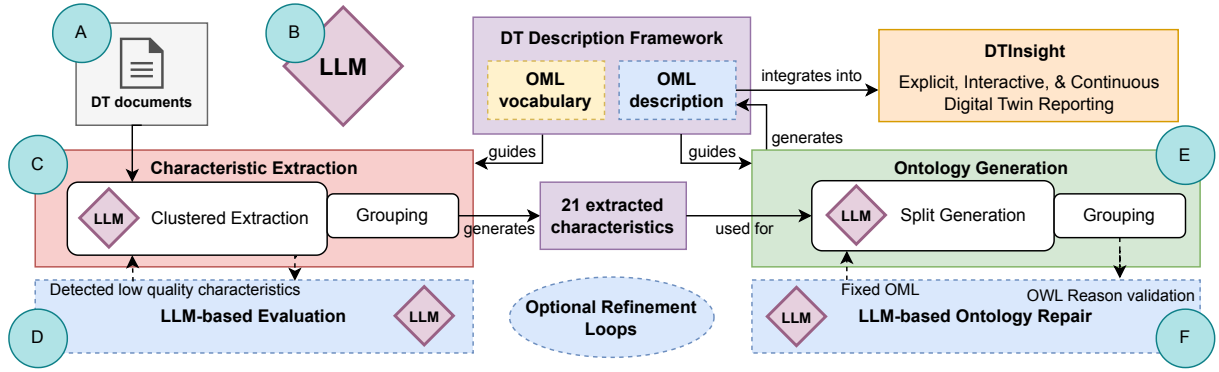


Fig. 2: Technical architecture of our DTscribe approach.

improve extracted characteristics, and an ontological reasoning loop (F) to repair and validate the generated OML.

Proposed Workflow. From the user’s perspective, the operation is a three-step workflow:

1. **Setup:** The user deploys the local LLM(s), the DTInsight tool, and our reporting component.
2. **Execution:** The user selects the relevant source artifacts (text files) describing the DT and initiates the reporting process.
3. **Output:** The system automatically generates a DT report aligned with the reporting framework, which is then visualized in DTInsight.

Our work is grounded in the incubator DT running example described in Section 2, providing sample input and output throughout this section.

3.1 Component A: DT Documents

Our pipeline supports the ingestion of standard text files, including `.docx` and `.txt`. However, for this article, we limit our processing to PDFs of academic papers that describe a DT exemplar, as listed in Section 4.1.

We have three assumptions about these documents: 1) they contain a description of a DT, 2) they do not explicitly use DTDF terminology so that our characteristic extraction is unbiased, and 3) they are devoid of malicious prompt injections that could tamper with our extraction process.

Document pre-processing

Raw user-provided documents are rarely suitable for direct ingestion by LLMs. This is particularly true for smaller models, which can easily

be sidetracked by formatting errors and superfluous information. As such, a pre-processing step is crucial to ensure the model receives a clean representation of the data.

To achieve this, we ingest each document and convert their text content into Markdown, preserving the original structural hierarchy of sections, headings, and lists. We automatically remove the “References” section from academic papers, as initial experiments indicated it degraded retrieval process quality. We preserve sentence and paragraph boundaries during chunking and embed the segments into a vector database for use in clustered characteristic extraction.

As detailed in Section 7.2, in future work we plan to expand this workflow to include multi-modal processing, such as converting figures to text to enhance retrieval context.

3.2 Component B: LLM

The LLM is the core component for two downstream tasks: (i) extracting a structured summary of DTDF characteristics and (ii) generating an OML description from this summary. The LLM component is treated as a configurable module, allowing us to experiment with different models, enabling systematic comparison without altering the pipeline logic.

Because no single model is optimal across both tasks required, we leverage six LLMs that vary in architecture, scale and deployment modality. These differences carry trade-offs in latency, cost and reliability; for instance, cloud hosted models can offer lower latency and access to higher parameter sizes, however their reliability depends on network stability and external service constraints.

On the other hand, locally available models allow for higher configuration and privacy controls, but remain limited by available hardware.

Since each task has its specific requirements (evidence-based and specific summarization for extraction, and strict syntactic/semantic validity for OML generation), these task-related constraints are considered alongside the practical trade-offs while reporting results for six model alternatives under a common experimental protocol. The final model choices are determined by the performance across both dimensions and the quality of outputs for both tasks.

3.3 Component C: Characteristic Extraction

This component extracts DTDF characteristics from user-provided documents with LLMs, which are then used for the subsequent OML generation.

3.3.1 Clustered Extraction with RAG

We employ a clustered extraction strategy to enhance the semantic alignment between the retrieved documents and the target characteristics. While extraction could technically be performed individually for each characteristic or monolithically for all twenty-one, our hypothesis is that clustering offers a distinct advantage in *retrieval precision* and *context management*.

By grouping semantically related characteristics, we can formulate more targeted retrieval queries (e.g., focusing solely on “Orchestration” terms rather than broad system descriptions). This ensures that the context provided to the LLM is dense with relevant evidence, minimizing the noise of unrelated text chunks. Furthermore, extracting a small cluster of items usually allows the model to maintain focus, reducing the hallucinations or omissions often observed when attempting to extract a large number of complex variables in a single pass.

We clustered the twenty-one characteristics based on semantic similarity by applying the k-means algorithm to the embeddings of their definitions (found in [4]), resulting in six groups: *Purpose* (CC_1), *Orchestration* (CC_2), *Components* (CC_3), *Connectivity* (CC_4), *Life-cycle* (CC_5), and *Governance* (CC_6). Table 7 (in the appendix) details the distribution of characteristics within

all clusters, along with their respective extraction prompts. For example, the *Orchestration* Cluster (CC_2) contains four characteristics: Twinning time-scale (C_7), Multiplicities (C_8), DT constellation (C_{12}) and Horizontal integration (C_{18}). An extracted description example for this cluster is showcased on Table 1.

Each cluster reuses a CoT prompt template (detailed in Listing 2), supplemented with the corresponding descriptions of characteristics to retrieve (detailed in Table 7) and semantically relevant document chunks retrieved using embeddings of tailored keywords (detailed in Table 8). The output is a structured JSON object mapping characteristic names to their extracted descriptions, which are then aggregated to serve as the input for the subsequent ontology generation component.

Prompting details

Our extraction prompt follows an instruction-driven structure inspired by CoT prompting to decompose the task [45]. As mentioned before, LLMs often struggle to reliably execute long, multi-constraint tasks in a single step. This component follows the same design principle of task decomposition adopted throughout the pipeline.

The prompt in Listing 2 has four main sections. First, it defines the role of the model as an expert in Digital Twin systems and ontology modeling, which encourages technical specificity and will guide the LLM towards domain-appropriate terminology acquired during its pre-training. Second, it provides the task inputs: (i) the retrieved context documents for the current cluster, (ii) the list of target characteristics along with their definitions, and (iii) optional judge feedback. Judge feedback is only included during refinement iterations (see Section 3.3.2); otherwise, this field is left empty.

Third, the prompt specifies a three-phase procedure. In the *reasoning phase*, the model is instructed to inspect the retrieved documents, recognize relevant evidence, and infer the use-case context, allowing for generalization across different domains. In the *extraction phase*, it must produce characteristic descriptions grounded solely in the provided documents, explicitly mark missing evidence with *Not in Document*, and incorporate judge feedback to refine low quality output. In the

Table 1: Example Characteristic Extraction of the Orchestration Cluster ($\mathcal{CC}_2$) by `gtp-oss:120b`, describing Twinning time-scale (C_7), Multiplicities (C_8), DT constellation (C_{12}) and Horizontal integration (C_{18}) for the incubator case study.

	Extracted Description
C_7	The DT synchronises with the Physical Twin through an event-driven RabbitMQ message bus. Sensor readings are streamed from the low-level driver to the broker and consumed by DT components (visualisation dashboard, Kalman filter, calibration module) in near-real-time, enabling sub-second latency for monitoring and control. Calibration can be triggered on-demand, and co-simulation can run on historical or live data streams.
C_8	The architecture defines a single Digital Twin instance associated with one Physical Twin (the incubator). The hierarchy consists of the Physical Twin, a low-level driver, a controller, and the Digital Twin which contains the simulation models, calibration and monitoring services. No decentralized or multiple-twin coordination is described.
C_{12}	The system follows a message-driven component architecture. Core components are: Low-level driver (sensor/actuator abstraction), Controller (bang-bang logic), Communication Server (RabbitMQ), Time-Series Database, Dashboard, Calibration service, Kalman-filter based anomaly detector, and co-simulation engine. RabbitMQ provides synchronization and decoupled communication; services subscribe to relevant topics, process data, and publish results, forming an orchestrated DT constellation.
C_{18}	Integration with external systems is achieved via the RabbitMQ message broker, which exposes standard AMQP interfaces for data exchange. Sensor data and control commands are published/subscribed over this broker, allowing other enterprise applications or additional Digital Twins to connect. The time-series database and dashboard consume the same streams, providing interoperability through common messaging protocols. No specific enterprise standards (e.g., OPC UA) are mentioned.

validation phase, the model is instructed to ensure that all statements are evidence-based, technically correct, complete according to the requested characteristics, and consistent with the judge feedback when provided.

Finally, strict formatting constraints are described, instructing the model must output only valid JSON, with no additional text. We enforce this constraint using LangChain’s structured output parsing, ensuring schema compliance and preventing additional content from appearing in the final extraction output.

3.3.2 Component D: LLM-based Evaluation

LLM generated extractions exhibit inherent variability due to sampling randomness and sensitivity to prompts. Consequently, the same characteristic might be correctly extracted in one run and fail in another. Therefore, an automated mechanism for assessing extraction quality is required to ensure that downstream OML generation operates on accurate and evidence-based inputs, increasing its success rate and accuracy.

To achieve this, we adopt an LLM-as-a-judge framework employing a secondary LLM. This evaluator examines each of the provided characteristics and provides feedback, approximating human judgment to guide refinement. To mitigate output randomness, we implement an iterative loop of *extraction*, *evaluation*, and *revision*. This process terminates when the characteristics achieve a sufficient quality score or when a maximum retry limit is reached. This type of refinement strategy pipeline employing repeated judgment–revision cycles has been shown to improve the reliability of LLM evaluations [53].

For the evaluation role, we employ `glm-4.7` as 1) large parameter size LLMs have been shown to provide higher-quality judgments [54]; and 2) utilizing a distinct model mitigates self-enhancement bias, a known risk when the same LLM performs both generation and evaluation [53]. While our current implementation utilizes a cloud-hosted evaluator to maximize judgment quality and improve reproducibility, the architecture is modular. Users prioritizing data sovereignty can substitute a local model or bypass the step entirely without disrupting the core infrastructure.

```

You are an expert in Digital Twin systems and ontology modeling.

You must extract a structured description for the following Digital Twin characteristics for a single Digital Twin system. For each characteristic, write one coherent description in English that is as specific and technical as possible and grounded ONLY in the provided documents. If the information for a characteristic is not available in the documents, return exactly the string "Not in Document" for that field.

CONTEXT DOCUMENTS:
{docs_content}

CHARACTERISTICS TO EXTRACT:
{description}

JUDGE FEEDBACK:
{judge_results}

INSTRUCTIONS:
1. REASONING PHASE: First, analyze the documents step by step:
  - Identify which documents contain relevant information for each characteristic
  - Note any technical details, technologies, or methodologies mentioned
  - Consider the specific context and domain of the use case

2. EXTRACTION PHASE: For each characteristic:
  - Provide specific, detailed descriptions based ONLY on the provided documents
  - Include concrete technical details (tools, technologies, protocols, methods)
  - Be precise about quantities, frequencies, and specifications when mentioned
  - If no evidence is found, state "Not in Document"
  - Implement the information from the judge feedback (if provided) to preserve high-quality characteristics and enhance characteristics scored less than 4.

3. VALIDATION PHASE: Review your extracted information:
  - Ensure all details come from the provided documents
  - Check that technical terms are used correctly
  - Verify completeness of the description
  - Ensure the extraction of characteristics is consistent with the judge feedback (if provided)

IMPORTANT: You MUST respond with ONLY valid JSON. Do not include any explanations, thinking tags, or additional text outside the JSON structure.

EXTRACTED CHARACTERISTICS (JSON FORMAT ONLY):
{format_instructions}

Remember: Be highly specific and technical. Include exact technologies, methods, and specifications mentioned in the documents. Return ONLY the JSON object with no additional text.

```

Listing 2: LLM prompt for clustered extraction.

Algorithm 1 outlines the process for sequentially evaluating and refining characteristics after each cluster extraction. It proceeds in four steps:

1. **Extract characteristics:** The LLM generates the initial set of characteristics extraction ($\mathcal{CC}'_i$) from the source documents (D) and the cluster of characteristics with their definitions ($\mathcal{CC}_i$).
2. **Evaluate quality and filter:** The judge assesses each characteristic on a 1–5 Likert scale (detailed in Listing 3). However, if the current iteration is a retry, the corresponding characteristics in P will be overwritten over the low-quality ones before judging the extraction ($\mathcal{CC}'_i$). Characteristics scoring below a threshold of 4 are isolated into a low-quality set (LQ), while high-quality items are preserved, added to the P set.
3. **Update sets:** If the current iteration is a retry, P and LQ sets are updated based on new E_q values, preserving the characteristics that reached an acceptable quality ($E_q(C) \geq 4$).
4. **Stop or refine:** If there are remaining low-quality characteristics ($LQ \neq \emptyset$) and there are still retries available, the loop will continue and the system attempts to refine the cluster by re-processing the extraction and preserving high quality items in the next iteration until the retry limit or acceptable quality is reached.

The evaluation score is guided by examples of Very Poor (1), Fair (3), and Very Good (5) characteristics, provided alongside the full source document to assess factual consistency [55], specificity and relevance. We enforce a strict JSON output format containing the characteristic name, reasoning, and score. This design ensures that judgments remain grounded solely in the provided evidence, reducing the risk of hallucinations.

3.3.3 Characteristic Extraction Summary

To summarize, Component C extracts twenty-one DTDF characteristics from user documents using RAG. This approach provides necessary context across six characteristic clusters, aiming for a high-quality extraction. The quality of these outputs is then iteratively refined by Component D (LLM-based Evaluation), which replaced descriptions scoring below a set threshold on a 1–5 Likert scale. The final output is a JSON object mapping

Algorithm 1 Iterative LLM-Based Refinement for One Cluster $\mathcal{CC}_i$

Require: Characteristics Cluster $\mathcal{CC}_i$, documents D , max retries $r_{\max}$, LLM

```

1:  $r \leftarrow 0$   $\triangleright$  retry count
2:  $P \leftarrow \emptyset$   $\triangleright$  preserved set
3:  $LQ \leftarrow \emptyset$   $\triangleright$  low quality set
4: while true do
  1. Extract characteristics
  5:  $\mathcal{CC}'_i \leftarrow \text{EXTRACT}(\mathcal{CC}_i, D)$ 
  2. Evaluate quality and filter
  6: if  $r > 0$  then
  7:  $\mathcal{CC}'_i \leftarrow (\mathcal{CC}'_i \setminus LQ) \cup P$ 
  8: end if
  9:  $\{E_q(C) \mid C \in \mathcal{CC}'_i\} \leftarrow \text{JUDGE}(\mathcal{CC}'_i, D)$ 
  10: if  $r = 0$  then
  11:  $LQ \leftarrow \{C \in \mathcal{CC}'_i \mid E_q(C) < 4\}$ 
  12:  $P \leftarrow \{C \in \mathcal{CC}'_i \mid E_q(C) \geq 4\}$ 
  3. Update sets
  13: else
  14: for each  $C \in LQ$  do
  15: if  $E_q(C) \geq 4$  then
  16:  $P \leftarrow P \cup \{C\}$ 
  17:  $LQ \leftarrow LQ \setminus \{C\}$ 
  18: end if
  19: end for
  20: end if
  4. Stop or refine
  21: if  $LQ \neq \emptyset$  and  $r < r_{\max}$  then
  22:  $r \leftarrow r + 1$ 
  23: continue
  24: end if
  25: break
  26: end while

```

each characteristic key to a textual description, typically spanning two to three sentences.

3.4 Component E: Ontology Generation

The goal of this framework component is to generate a valid OML description from the set of natural language DTDF characteristics using an LLM. To do so, it a) splits characteristics for OML instance generation, and b) runs an iterative LLM-based ontology repair loop.

1 - Very poor
- "Not in Document" or states the characteristic is missing when the source documents DO mention relevant information.
- Vague or generic information.
- Uses speculative language such as "likely", "possibly" instead of directly using the text.
- No relevance to the specific system mentioned in the Source Documents.
2 - Poor
- Very general definition.
- Not specific to the system mentioned in the Source Documents.
- Lacks technical content.
3 - Fair
- Relevant to the specific system.
- Some correct context, but missing key technical details that are present in the Source Documents.
4 - Good
- Correct and specific.
- Contains relevant technical details.
- Could still be expanded with more depth.
5 - Very good
- "Not in Document" or states the characteristic is missing and the Source Documents DO NOT contain relevant information.
- Highly specific and detailed.
- Provides in-depth technical explanation of the characteristic's behavior within the system.
- Strongly grounded in the provided sources.

Listing 3: LLM characteristic scoring scale guide.

3.4.1 Split Generation

Smaller LLMs struggle to maintain structural coherence. As such, their context window must be optimized to minimize context noise that could disrupt the current LLM objective. We thus propose *split generation* of the characteristics by only tasking the LLM to generate OML instances that cannot just be constructed programmatically. This aims to reduce context length, lower inference cost, and simplify the generation task.

As such, we categorized the DTDF characteristics into a) *component characteristics* $C_{\text{component}}$, which are multi-attribute, interdependent, and composed of multiple instances, and b) *description characteristics* $C_{\text{description}}$, which are single-attribute and independent. Component-based OML instances are LLM-generated, guided by characteristics, while simple description-based ones are programmatically constructed.

For example, C_{10} and C_{11} are component characteristics that respectively describe the models/-data and tooling/enablers in the system. These are represented by the boxes connected by arrows in

Figure 1. Oftentimes, data or models are used by those tools or enablers, which make them adhere to the definition of a component characteristic.

Conversely, description characteristics like C_9 describe the life-cycle stages of the DT and system, which have no representation in our DT constellation. Thus, C_9 is best kept as a textual-only description in the characteristic table report. Other examples of description characteristics are shown in Table 1.

Equation (1) showcases the two characteristic sets and their assigned characteristics.

$$\begin{aligned} C_{\text{component}} &= \{C_2, C_3, C_4, C_6, C_{10}, C_{11}, C_{17}\}, \\ C_{\text{description}} &= \{C_1, \dots, C_{21}\} \setminus C_{\text{component}}. \end{aligned} \quad (1)$$

Simply providing the LLM with the DTDF OML vocabulary resulted in too many syntactic and semantic mistakes, especially at lower parameter size. As such, we employed CoT prompting to provide the LLM with curated rules and guides. We modified the prompt until our preliminary experiments gave results of sufficiently-high ontology generation quality. The prompt provides a list of characteristics to extract or not, naming conventions, a syntax guide, rules for generating the OML instances, and a characteristic validation checklist. Listing 4 showcases the LLM syntax guide provided to the LLM to generate the component characteristics.

3.4.2 Component F: LLM-based Ontology Repair

Despite the constrained generation prompts, the initial OML output rarely satisfies all syntactic and semantic constraints. Common errors include *hallucinated references* (linking to instances that were never defined), *type mismatches* (assigning a property to the wrong entity type), and *syntax violations* (malformed OML structures). Rather than discarding these outputs or requiring manual user intervention, we leverage the deterministic error reporting of an ontology reasoner to guide the LLM in a self-correction loop [35].

Algorithm 2 details this iterative generation and repair process:

1. **Split Generation:** The system programmatically constructs the description ontology ($\mathcal{O}_{\text{desc}}$) from $C_{\text{description}}$ and prompts the LLM

```
// C2: Acting Components
instance <ActuatorName> : DTDFVocab:ActingComponent [
  base:desc "<specific description from
characteristics>"
]
// C3: Physical sensing components
instance <SensorName> : DTDFVocab:SensingComponent [
  base:desc "<specific description from
characteristics>"
  DTDFVocab:producedData <DataTransmittedName>]
// C4: Physical-to-virtual interaction
instance <DataTransmittedName> :
  DTDFVocab:DataTransmitted [
    DTDFVocab:producedFrom <SensorName1>,
    <SensorName2>]
// C10: Models/Data
instance <ModelName> : DTDFVocab:Model [
  base:desc "<specific model description from
characteristics>"
  DTDFVocab:inputTo <EnablerName1>, <EnablerName2>]
instance <DataName> : DTDFVocab:Data [
  base:desc "<specific data description from
characteristics>"
  DTDFVocab:inputTo <EnablerName1>, <EnablerName2>
  DTDFVocab:fromData <DataTransmittedName>]
// C11: Enablers
instance <EnablerName> : DTDFVocab:Enabler [
  base:desc "<specific enabler description from
characteristics>"
  DTDFVocab:enables <ServiceName1>, <ServiceName2>]
// C6: Services
instance <ServiceName> : DTDFVocab:Service [
  base:desc "<specific service description from
characteristics>"
  DTDFVocab:provides <InsightName1>, <ActionName2>]
// C17: Insights/Actions
instance <InsightName> : DTDFVocab:Insight [
  base:desc "<specific insight description from
characteristics>"
]
instance <ActionName> : DTDFVocab:Action [
  base:desc "<specific action description from
characteristics>"
  DTDFVocab:IsAutomatic <true|false>]
```

Listing 4: LLM syntax guide for OML generation of the DTDF description model using component characteristics.

- to generate the component ontology ($\mathcal{O}_{\text{comp}}$) from $C_{\text{component}}$.
2. **Integration and Validation:** The system merges these fragments and executes the OWL Reason⁶ ontology reasoner. This tool validates type satisfiability, logical consistency, and reference integrity [40]. If validation succeeds, the ontology is deployed to the Fuseki server. If validation fails, the system triggers the error injection phase.
3. **Error Injection and Repair:** To facilitate repair, we capture the reasoner’s output and inject the specific error messages directly into the OML description model as context-aware

⁶<https://github.com/opencaesar/owl-tools/tree/master/owl-reason>

Algorithm 2 Iterative Refinement of LLM-Generated OML Descriptions

Require: Characteristics $C_{\text{component}}$, $C_{\text{description}}$,
Max retries R_{max}

Ensure: Validated Ontology $\mathcal{O}$ or Failure

```

1. Split Generation
1:  $\mathcal{O}_{\text{desc}} \leftarrow \text{STATICGEN}(C_{\text{description}})$ 
2:  $\mathcal{O}_{\text{comp}} \leftarrow \text{LLMGEN}(C_{\text{component}})$ 
3: for  $r \leftarrow 0$  to  $R_{\text{max}}$  do
2. Integration and Validation
4:    $\mathcal{O} \leftarrow \text{INTEGRATE}(\mathcal{O}_{\text{comp}}, \mathcal{O}_{\text{desc}})$ 
5:    $(\text{valid}, \text{report}) \leftarrow \text{REASONERCHECK}(\mathcal{O})$ 
6:   if valid then
7:      $\text{DEPLOYTOFUSEKI}(\mathcal{O})$ 
8:     return  $\mathcal{O}$ 
9:   end if
3. Error Injection and Repair
10:  if  $r = R_{\text{max}}$  then
11:    return Failure
12:  end if
13:   $\mathcal{O}_{\text{comp}} \leftarrow \text{INJECTERRORS}(\mathcal{O}_{\text{comp}}, \text{report})$ 
14:   $\mathcal{O}_{\text{comp}} \leftarrow \text{LLMREPAIR}(\mathcal{O}_{\text{comp}}, C_{\text{component}})$ 
15: end for

```

comments immediately following the problematic lines. For example, if the model references an undefined entity `SensorX`, the line is annotated with `// TODO: Fix the error on this line using the following error message: 'Couldn't resolve reference to NamedInstance 'SensorX''`. Other reasoner outputs (such as the `reasoning.xml` file), are directly ingested into the prompt. The LLM is then prompted to repair the code using these annotations as a guide. This cycle repeats until the ontology is valid or the maximum retry limit (R_{max}) is exhausted.

Listing 5 shows an error-injected OML description. The first error misspells *Sensing-Component* as *SensorComponent*, the second couldn't link to *ThermalSensorDataTransmitted* because its current instance is not related to a vocabulary concept, and the third error misspells the actual instance name (*ThermalSensor*).

3.4.3 Ontology Generation Summary

Component E generates an OML description model compatible with DTInsight using the

```

instance ThermalSensor : DTDFVocab:SensorComponent [
  // TODO: Fix the error on this line using the
  // following error message: 'Couldn't resolve
  // reference to Entity 'DTDFVocab:SensorComponent'.'
  base:desc "Thermal sensors measuring system
  temperature for monitoring",
  DTDFVocab:producedData
  ThermalSensorDataTransmitted // TODO: Fix the
  // error on this line using the following error
  // message: 'Couldn't resolve reference to
  // SemanticProperty
  // 'ThermalSensorDataTransmitted'.'
]
instance ThermalSensorDataTransmitted :
  DTDFVocab:DataTransmitted [
  DTDFVocab:producedFrom ThermalSensorComponent //
  // TODO: Fix the error on this line using the
  // following error message: 'Couldn't resolve
  // reference to NamedInstance
  // 'ThermalSensorComponent'.'
]

```

Listing 5: Invalid DTDF OML description model excerpt with injected reasoner errors.

extracted characteristics. To minimize the LLM's context window and improve quality, we split the data: component characteristics are generated via LLM, and description characteristics are generated programmatically.

We then employ an iterative ontology repair loop (component F) using ontological reasoning to identify errors in the OML description. These errors are fed back to the LLM for correction until the ontology achieves validity or the maximum retry threshold is reached.

The generated OML description model resembles Listing 5 (without errors) and is ingested by DTInsight to create a constellation similar to the one showcasing the incubator on Figure 1. As described in our work [7], DTInsight queries the OML description model to build component connections and extract the textual descriptions for the reporting page.

3.5 Approach Summary

To answer *RQ1*, we propose DTscribe, a modular technical architecture designed to support local LLM deployment for the secure and automated processing of DT documentation. As illustrated in Figure 2, the architecture is composed of a three-stage pipeline:

1. **Ingestion and Pre-processing (Component A):** Raw documentation is converted to Markdown and chunked for RAG, creating a clean context for downstream tasks.

2. **Characteristic Extraction (Components B, C, D):** We employ a *clustered extraction strategy*, grouping the twenty-one DTDF characteristics into six semantic clusters to maximize retrieval precision. This process is reinforced by an optional *LLM-as-a-judge* refinement loop, where a secondary model evaluates extraction quality on a Likert scale and triggers re-generation for low-scoring descriptions.
3. **Ontology Generation (Components E, F):** To produce valid OML models, we utilize a *split generation* approach. Complex component instances are generated by the LLM via CoT prompting, while simple descriptive attributes are constructed programmatically. This phase includes an iterative *ontology repair loop*, where a symbolic reasoner validates the output and injects error reports back into the LLM context to guide self-correction.

By integrating local LLMs with symbolic verification (reasoners) and iterative refinement loops, this architecture enables the extraction of DT characteristics and the generation of syntactically valid OML description models to create a visualization in DTInsight.

4 Assessing Approach Quality

This section answers *RQ2* by a) describing the input papers used to evaluate our approach, including the definition of a *partial ground truth* for characteristic extraction, and then b) defining metrics for LLM-generated textual reports. Together, these elements will be used to answer *RQ3* in the evaluation section (Section 5).

In particular, we use two sets of metrics for evaluating our approach decisions and the produced artifacts: *extraction metrics* (Section 4.3) for characteristic extraction, and *ontology metrics* (Section 4.4) for ontology generation. In both cases, we track LLM generation time.

4.1 DT Exemplar Papers

We benchmark our approach against five DT paper exemplars from the literature:

- **P1:** *A Digital Shadow for Accurate Robot Motion Control: Integrating Data with Friction Models* [56].

- **P2:** *Engineering Automotive Digital Twins on Standardized Architectures: A Case Study* [25].
- **P3:** *Lab-Scale Gantry Crane Digital Twin Exemplar* [57].
- **P4:** *Model-driven Digital Twins for AECO* [58].
- **P5:** *The Incubator Case Study for Digital Twin Engineering* [41].

P1 to **P4** were recently published at the 2nd International Conference on Engineering Digital Twins (*EDTconf 2025*)⁷, and thus provide high quality DT exemplars for our pipeline. We use **P5** as our case study as it presents a recurring example throughout previous papers. Despite **P1** to **P4** being written after the DTDF conceptualization, none of them explicitly use the description framework. All these aspects mean the papers are well-suited for evaluating our approach.

4.2 Establishing Partial Ground Truth

To determine characteristic extraction accuracy, we established a partial ground truth by having all three authors of this article independently rate the presence of twenty-one characteristics across the five DT exemplars used for evaluation. The results are compiled in Figure 3.

To validate the reliability of our partial ground truth, we calculated the inter-rater reliability (IRR) among the three raters. We obtained a Fleiss' Kappa score of $\kappa = 0.625$, which indicates *substantial agreement* [59, 60].

Finally, we aggregated the ratings using a majority voting scheme: a characteristic was categorized as *present* if at least two raters agreed on its presence, and *absent* otherwise. We use these aggregated labels to determine the extraction accuracy in Section 4.3.1.

4.3 Extraction Metrics

We evaluate the extracted characteristic descriptions using two metrics: *extraction accuracy* and *extraction quality*.

Formal Notation: Let $D = \{d_1, \dots, d_n\}$ be the set of n user-provided input documents. For $k \in \{1, \dots, 21\}$, denote the k -th characteristic of the DTDF extracted by the model. The value of C_k is either a textual description or NID (*Not in*

⁷<https://conf.researchr.org/home/edtconf-2025>

Digital Twin Characteristic Presence Consensus

Aggregation of 3 Raters

Paper ID	C1: System under study	C2: Physical acting components	C3: Physical sensing components	C4: Physical-to-virtual interaction	C5: Virtual-to-physical interaction	C6: DT services	C7: Twinning time-scale	C8: Multiplicities	C9: Life-cycle stages	C10: DT models and data	C11: Tooling and enablers	C12: DT constellation	C13: Twinning process and DT evolution	C14: Fidelity and validity considerations	C15: DT technical connection	C16: DT hosting/deployment	C17: Insights and decision making	C18: Horizontal integration	C19: Data ownership and privacy	C20: Standardization	C21: Security and safety considerations
P1	3	0	3	3	0	3	3	2	3	3	3	3	2	2	3	1	3	1	0	3	0
P2	3	3	3	3	3	3	3	2	3	3	3	3	2	2	3	2	3	0	0	3	2
P3	3	3	3	3	3	3	1	2	3	3	3	3	1	3	3	2	3	0	0	1	0
P4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	2	0	0	3	1
P5	3	3	3	3	3	3	3	2	1	2	3	0	1	3	3	2	3	0	0	0	0

Consensus Level
 Absent (0/3 votes) Weak Presence (1/3 votes) Moderate Presence (2/3 votes) Strong Presence (3/3 votes)

Fig. 3: Consensus for DTDF characteristic presence. Papers IDs defined in Section 4.1.

Document), indicating the LLM did not extract the characteristic from *D*.

4.3.1 Extraction Accuracy

Extraction accuracy assesses the system’s ability to correctly identify the presence or absence of characteristics in the source documents (avoiding hallucinations).

We measure the alignment between the LLM and human annotators regarding the presence of characteristics within the documents based on this *partial ground truth*. Extraction is treated as a binary classification task for each of the twenty-one characteristics, ignoring the textual quality of the output. Let $\mathbf{p} \in \{0,1\}^{21}$ be the binary ground-truth vector derived from human assessment, and let $\mathbf{t} \in \{0,1\}^{21}$ be the binary LLM-judged extraction vector. Their components are defined as:

$$p_k = \begin{cases} 1 & \text{if characteristic } k \text{ is present in } D \\ & \text{(human label),} \\ 0 & \text{otherwise,} \end{cases}$$

$$t_k = \begin{cases} 1 & \text{if } C_k \neq \text{NID (any textual description),} \\ 0 & \text{if } C_k = \text{NID.} \end{cases}$$

The extraction accuracy E_{Acc} is calculated as the proportion of concordant pairs between the two vectors:

$$E_{\text{Acc}} = \frac{\text{Agreement count}}{21} = \frac{1}{21} \sum_{k=1}^{21} \mathbf{1}(p_k = t_k)$$

where $\mathbf{1}(\cdot)$ is the indicator function.

4.3.2 Extraction Quality

Extraction quality evaluates the semantic precision, relevance, and formulation of extracted descriptions via an LLM-based assessment. Specifically, it measures adherence to the scoring criteria defined in Listing 3.

Unlike extraction accuracy, extraction quality is a *reference-free* metric. This approach is necessary because DT reports must serve stakeholders with heterogeneous requirements, ranging from high-level summaries to distinct technical specifications. Consequently, enforcing a single “ground truth” reference summary would penalize valid details essential to specific user profiles.

We employ an evaluator LLM to rate each characteristic description C_k given the source document D . We define the scoring function $E_q(C_k, D) \in \{1, \dots, 5\}$ as follows:

- If C_k contains extracted text, the LLM assigns a score on a 5-point Likert scale (1=lowest quality, 5=highest quality) based on evidence present in D .
- If $C_k = \text{NID}$ (Not in Document), the score is binary: 5 if the absence is verifiable in D , and 1 if the omission is incorrect (hallucinated absence).

The evaluator is prompted with specific criteria and few-shot examples. For every characteristic, the model generates both the score E_q and a textual rationale to ensure interpretability.

As detailed in the discussion (Section 7), we utilize this metric primarily as a quality enhancement signal within the LLM refinement loop; therefore, it is not reported in the results (Section 6).

4.4 Ontology Generation Metrics

We evaluate the pipeline using *success rates* for ontology generation. These metrics quantify the likelihood of the LLM producing a valid OML description at each retry step, as well as the cumulative number of retries required to converge. To determine validity, we use the OWL Reasoner⁸ to detect syntactic and logical errors.

Formally, we model the generation as a sequential process where a reasoner error triggers a repair loop up to a maximum of M retries. Let the random variable $T \in \{0, 1, \dots\}$ denote the retry index of the first successful validation. $T = 0$ represents immediate success, while $T > M$ indicates that the process failed and no valid ontology was generated within the limit M . Based on T , we define three complementary metrics:

Unconditional Success Rate (S_i): This measures the probability that a random generation run yields a valid result exactly at retry step i . It represents the distribution of success across steps:

$$S_i = \Pr(T = i) \in [0, 1]$$

Cumulative Success Rate (R_i): To assess system reliability up to a given budget, we measure the total probability of obtaining a valid ontology

by step i . This metric is monotonically increasing:

$$R_i = \Pr(T \leq i) = \sum_{j=0}^i S_j \in [0, 1]$$

Conditional Success Rate (H_i): To evaluate the repair mechanism independently of previous runs, we measure the probability of success at step i , given that the run has not yet succeeded. While S_i and R_i provide a global view, H_i offers a local view of the repair efficiency at a specific iteration:

$$H_i = \Pr(T = i \mid T \geq i) \in [0, 1]$$

5 Evaluation

This section details the evaluation of our approach. In particular, we recall our research questions and introduce our experimental setup.

Our evaluation is guided by the work of Baltes *et al.* [61] by ensuring we follow recommended designing and reporting guidelines of empirical studies using LLMs.

We recall our research questions:

RQ1: What is a **technical architecture with local LLMs** to enable the extraction of DT characteristics and the generation of valid OML description models? *Answered in Section 3.5.*

RQ2: How can **LLM-generated textual reports be evaluated** in terms of extracting complete and high-quality DTDF characteristics? *Answered in Section 4.*

RQ3: How do **choices made in our approach improve the quality and validity of the generated artifacts**, such as LLM-based evaluation, ontology-based repair, and LLM selection? *The current section details the experiments used to correlate quality improvements with approach choices.*

In particular, we investigate the following in RQ3:

RQ3.1: Local LLM capability

RQ3.2: Impact of clustering and LLM judging

RQ3.3: Effectiveness of iterative OML repair

⁸<https://github.com/opencaesar/owl-tools/tree/master/owl-reason>

5.1 Experimental Setup

As recommended by [61] in Section 4.2 “Report Model Version, Configuration, and Customizations”, we describe our experimental setup for transparency and reproducibility.

Software Setup

We evaluated six open-source LLM configurations with varying parameter count: `ministral-3(:3/8/14B)`⁹, `qwen3-next(:80B)`¹⁰, and `gpt-oss(:20/120B)`¹¹. Vector embeddings were generated using `embeddinggemma`¹², selected for its state-of-the-art performance among multilingual models under 500M parameters on the Massive Text Embedding Benchmark (MTEB) [62]. Additionally, we used the open-source `glm-4.7`¹³ cloud model as a judge. LLM deployment characteristics are listed in Table 2.

The software environment combined Ollama¹⁴ for local inference, LangChain¹⁵ for orchestration, and ChromaDB¹⁶ for vector storage and retrieval. Together these tools form a modular workflow that, apart from the LLM judge, utilizes models of reasonable size for practitioners for local deployment on work computers (under 20B parameters) or servers (80B and 120B parameters).

LLM Hyperparameters

While the ontology repair loop may benefit from the diversity introduced by default sampling parameters, we argue that the evaluation mechanism must remain strictly stable to serve as a reliable benchmark. As such, we use default LLM hyperparameters (temperature, top-k, top-p) for characteristic extraction and ontology generation. To enforce determinism in the LLM judge, we set its temperature to 0 and fix the seed, aligning with practices from domain modeling [19, 63].

Hardware Setup

All experiments were executed on a Windows 11 laptop with 32 GB of RAM and an NVIDIA

GeForce RTX 4060 8GB VRAM GPU, with LLMs running on Ollama Cloud¹⁷ to parallelize experimentation for this article.

5.2 Experimental Process

To quantify the impact of our architectural components [35], we conducted an ablation study by systematically isolating three core features:

1. **Context Management:** We bypassed the clustering module, ingesting the entire document directly into the LLM context to evaluate the performance of organized data retrieval.
2. **Iterative Feedback:** We disabled the LLM-as-a-judge refinement loop to evaluate raw extraction accuracy and quality, and the resulting OML generation success rates.
3. **Convergence Efficiency:** We analyzed the first-pass success rate by isolating results obtained prior to the OML retry logic. This allows us to measure the base LLM ability to satisfy OML constraints on the first attempt, thereby identifying the improvements provided by the subsequent refinement loops.

This resulted in four experiment configurations: **Base** with full document ingestion into context, and no judge; **+Cluster** with clustered extraction with RAG and no judge; **+Judge** with full document ingestion into context and the LLM-as-a-Judge refinement loop; **+Cluster+Judge** with both clustered extraction with RAG and the LLM-as-a-Judge refinement loop. For each configuration, we isolate success rates obtained before and after the OML refinement loop.

We ran each configuration twenty-five times ($n = 25$) to account for the variability of LLM responses. We leverage Ollama Cloud’s infrastructure to execute multiple experiments concurrently, but both **Base** and **+Cluster** configurations could be ran fully locally by deploying `ministral-3(:3B/8B)` directly on our machine.

Our experimental pipeline was executed three thousand times in total, and ran for around five days. This highlights the necessity to use a cloud infrastructure to parallelize computation and do experiments at scale, even for local models. We also had to develop several strategies to ensure

⁹<https://ollama.com/library/ministral-3>

¹⁰<https://ollama.com/library/qwen3-next>

¹¹<https://ollama.com/library/gpt-oss>

¹²<https://ollama.com/library/embeddinggemma>

¹³<https://ollama.com/library/glm-4.7>

¹⁴<https://ollama.com/>

¹⁵<https://www.langchain.com/>

¹⁶<https://www.trychroma.com/>

¹⁷<https://ollama.com/cloud>

Table 2: LLM deployment characteristics. Category size is representative for use in our analysis. Note that the VRAM requirement may be higher in practice due to context overhead and model activations.

Model	Parameter count	Precision	Min. VRAM	Size Category	Release Date
embeddinggemma	0.3B	BF16	~0.6GB	Small	2025-09-04
ministral-3	3B	INT4	~3GB	Small	2025-12-02
ministral-3	8B	INT4	~6GB	Small	2025-12-02
ministral-3	14B	INT4	~9GB	Small	2025-12-02
gpt-oss	20B	INT4	~16GB	Medium	2025-08-05
qwen3-next	80B	INT4	~50GB	Medium	2025-09-10
gpt-oss	120B	INT4	~80GB	Large	2025-08-05
glm-4.7	358B	INT4	~200GB	Large	2025-12-22

architecture robustness: mitigating network failures from Ollama Cloud servers, handling file locks for concurrent access to the OWL Reasoner, and detecting missing experiment configurations by assigning IDs and tracking experiment runs.

6 Results

In this section, we present the results of our evaluation. Specifically, we address the three sub-RQs of RQ3 to determine the impact of various features and configurations within our pipeline.

We have created a reporting page¹⁸ to showcase the end result of our pipeline, ingested by DTInsight [7]. We specifically hand-picked an OML description file from a **+Judge** configuration for **P2** with **gpt-oss:20B** as Table 5 demonstrates this setup contains the most accurate extractions (around 99% accuracy).

6.1 RQ3.1: Local LLM Capability

We first investigate the relationship between model size, extraction quality, and ontology generation quality. Our primary goal is to determine whether **baseline local LLMs** can produce adequate results without relying on additional pipeline enhancements.

Extraction Accuracy

Focusing on the extraction accuracy of our **Base** configuration, Table 5 reveals that all baseline LLMs, including the small **ministral-3:3B**,

achieved an extraction accuracy above 70%. Second, larger models generally yielded better results, as expected. Third, specific papers, such as **P5**, present inherent extraction difficulties across all LLMs regardless of size.

We present an alternative view of these results in Figure 5, plotting extraction accuracy against model size. While the data is noisy and combines non-baseline configurations, it suggests that scaling model size improves accuracy in most cases. However, the performance dips for **P4** (red diamond) and **P5** (purple downwards triangle) indicate that larger models do not guarantee improvement. This is likely due to missing information in the underlying papers themselves, as indicated by our partial ground truth (Figure 3).

Surprisingly, contrary to the hypothesis that smaller models require RAG to mitigate input complexity, baseline local models with as few as 3B parameters achieved robust extraction accuracy. Notably, these smaller models frequently performed within five points of our largest model.

Ontology Generation

To determine baseline generation effectiveness, we evaluate the models’ ability to output valid OML models. The comprehensive results are provided in Table 6, with the peak success rate for each model plotted in Figure 6. In this figure, empty circles represent the success rate on the first round of OML generation (prior to reasoner-based refinement), while filled circles indicate the success rate after four rounds of refinement.

For OML generation on the first try, Table 6 shows all models except the largest one struggle with around sub-40% success rate on first try.

¹⁸ Available at <https://oakeslabmtl.github.io/dtdf-dtsubscribe/>

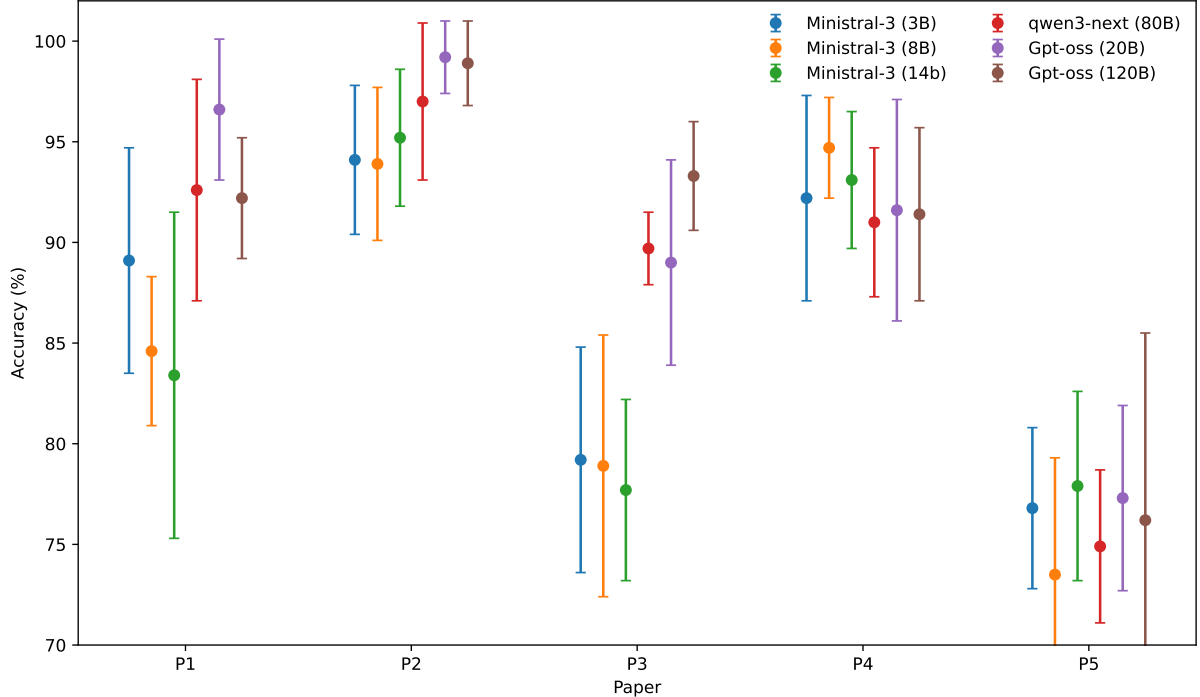


Fig. 4: Best *extraction accuracy* results obtained across papers and models.

Only the largest models achieves around 60% success. Figure 7 specifically shows for paper **P5** that the success rate of the **Base** configuration’s first attempt increases with model size, but generally remains low. Section 6.3 discusses the OML generation success after refinement loops.

We also captured the instance count for the OML models generated. To briefly summarize, most configurations and models produced between 25 to 50 instances, with roughly equivalent variance. Thus, we draw no insights from this analysis.

RQ3.1 Summary

Small to medium baseline models can be as suitable in terms of extraction accuracy (above 70%) compared to larger models, depending on the paper.

However, these small to medium models consistently struggle (sub-40% success) to generate OML description models on the first try. Only our largest model is good enough for OML generation without refinement loops, at around 60% success.

6.2 RQ3.2: Impact of Clustering and LLM Judging

Having established baseline model capabilities, we then evaluate the impact of our pipeline enhancements. In Table 5, we provide the full extraction accuracy results by configuration. These results indicate that, contrary to our expectations, **extraction configurations involving clustering (with RAG) or judging provide limited accuracy gains relative to their additional computational cost.**

Table 3: Average extraction accuracy percentage *change* (%) for the **+Judge** and **+Cluster** configuration compared to **Base**.

Config.	Overall	P1	P2	P3	P4	P5
+Judge	2.11	1.53	2.83	1.25	4.20	0.72
+Cluster	-18.51	-31.19	-22.95	-13.15	-14.59	-10.65

Adding a judging step (**+Judge**) provides the highest extraction accuracy across the majority of models. This means the judge model successfully

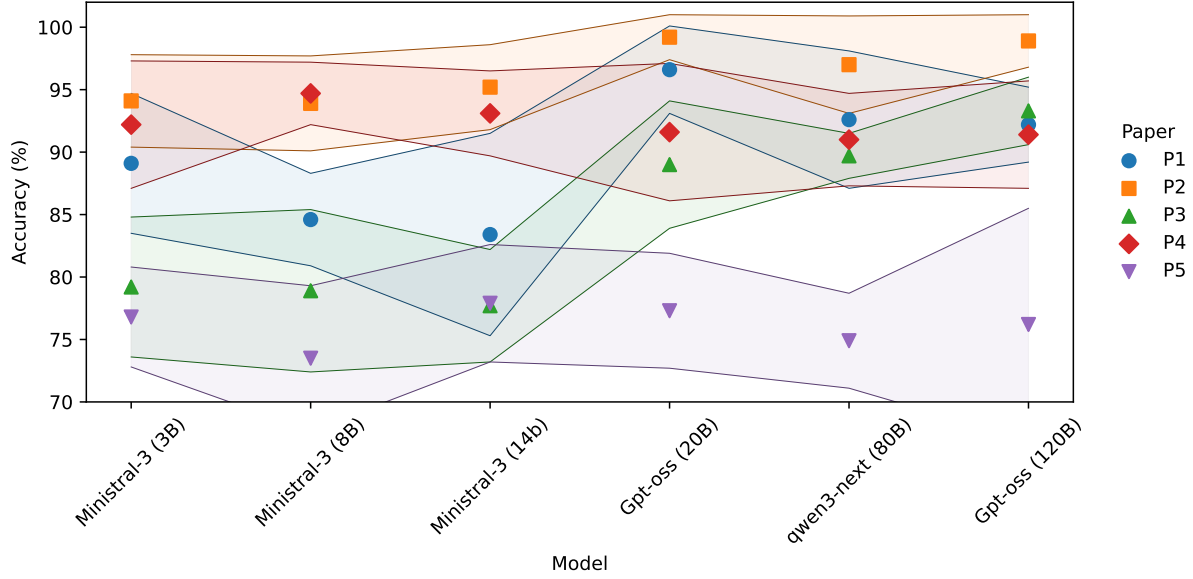


Fig. 5: Extraction accuracy with the error spread region, plotted across models.

Table 4: Average processing time for extraction and OML generation (four maximum retries) across configurations.

Configuration	Extract Time (s)	OML Gen. Time (s)	Total Time (s)
Base	62.5	255.0	317.5
+Cluster	208.2	195.8	404.0
+Judge	647.9	257.2	905.1
+Cluster+Judge	2636.1	213.8	2849.9

identifies low-quality characteristics to prompt corrections. However, as shown in Table 3, accuracy only improves by an average of 2.11% and comes at a steep computational price.

As detailed in Table 4, the **+Judge** configuration increases extraction processing time by an order of magnitude compared to the **Base** configuration, almost tripling the total experiment duration (from 317.5s to 905.5s). Combining these methods in the **+Cluster+Judge** configuration creates a bottleneck by increasing processing time by a factor of 9 to over 45 minutes per run, representing the least efficient trade-off. *Note: These results rely on parallelized experiments with LLMs running in the cloud, meaning the absolute processing times should be interpreted as relative*

estimates of computational efficiency rather than precise benchmarks for local execution.

Contrary to our initial expectations, clustered extraction (**+Cluster**) severely degraded performance, returning the lowest accuracy across the board (-18.51% overall, as shown in Table 3). This degradation is most pronounced in larger, more capable models (**qwen3-next**, **gpt-oss**), which surprisingly underperform compared to smaller models (**ministral-3**) in this configuration. We hypothesize that clustering fragments the semantic context: while smaller models may hallucinate or guess over gaps, larger models strictly adhere to the provided (fragmented) context, leading to valid but unhelpful “Not in Document” outputs.

RQ3.2 Summary

The LLM-as-a-judge loop improves accuracy by a marginal average of 2.11% but triples processing time. Conversely, clustering fragments document context and degrades accuracy by an average of 18.51%, hindering larger models. The combination of both methods is infeasible due to significant increases in processing time.

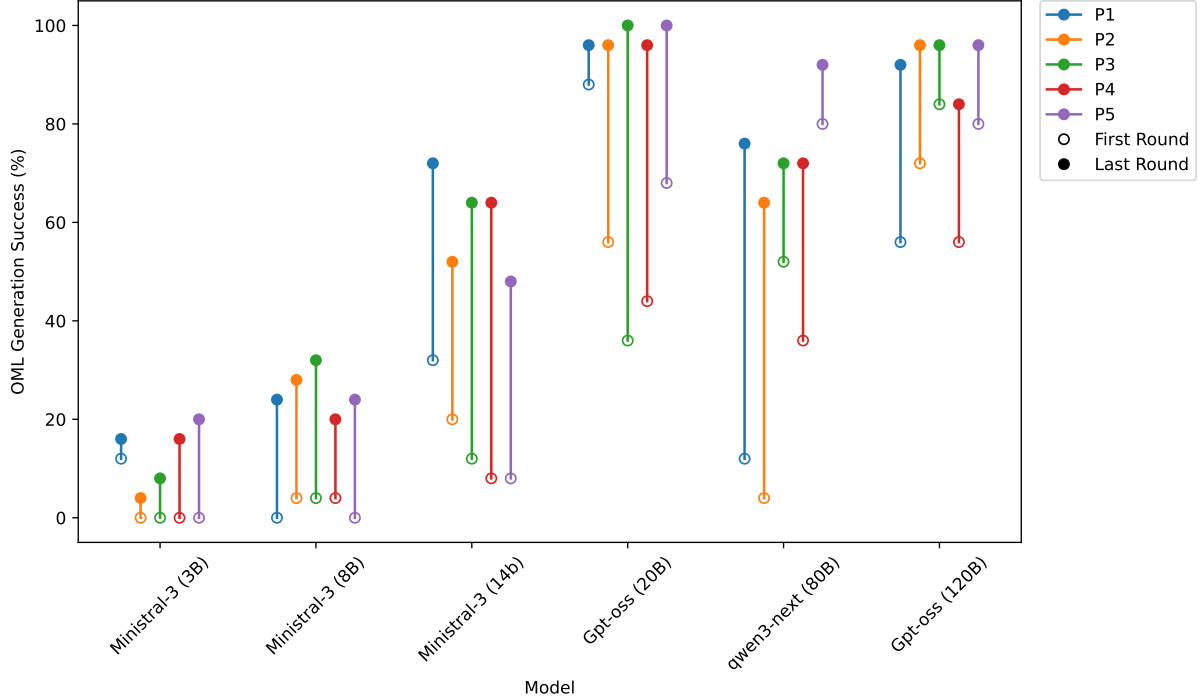


Fig. 6: Best OML generation success for each model after four retries.

6.3 RQ3.3: Effectiveness of Iterative OML Repair

As detailed in Table 6, the OML generation results demonstrate that **iterative repair mechanisms improve OML generation, allowing medium-sized models to compete with larger counterparts.**

Impact of model size. Retry-based repair loops improve cumulative success rates with four retries (R_4) over first-attempt performance (S_1), though the utility tends to peak at a certain model size and capability. Smaller models (ministral-3:3B/8B/14B) lack the baseline capability to effectively leverage the repair mechanism. The largest model (gpt-oss:120B) shows diminishing returns (19.0% improvement) due to its already high initial success rate ($\sim 60\%$). Only medium- to large-sized models (gpt-oss:20B, qwen3-next:80B) truly benefit from this mechanism with large improvements (33–39%). We hypothesize that small differences between these two models emerge from base LLM capabilities and response to prompt structure, which overshadow size differences (20B vs. 80B parameters).

Number of retries needed. Figure 7 illustrates the optimal retry thresholds. The largest model maximizes its benefit after just one or two retries. Medium-sized models (gpt-oss:20B, qwen3-next:80B) require two to three retries to reach their peak success rates. Smaller models show little to no benefit from the retry mechanism at all regardless of the number of attempts.

Impact of extraction configuration. The effectiveness of repair sometimes depends on extraction accuracy: a low accuracy (as is the case of medium to large models as shown in Table 6 for the +Cluster configuration) leads to low task complexity, as most of the extracted characteristics are actually *Not in Document*. This explains the high success rates for those models (Figure 7).

Table 5: Extraction accuracy ($E_{\text{Acc}}(\%)$, $n = 25$), with two judge retries. **Blue** denotes best. Papers IDs defined in Section 4.1.

Paper ID	Configuration	ministral-3 (3B)	ministral-3 (8B)	ministral-3 (14B)	qwen3-next (80B)	gpt-oss (20B)	gpt-oss (120B)
P1	Base	85.7 $\pm$ 7.7	84.6 $\pm$ 3.7	82.7 $\pm$ 7.4	90.5 $\pm$ 6.0	96.2 $\pm$ 5.3	90.1 $\pm$ 4.1
	+Cluster	73.5 $\pm$ 10.5	73.9 $\pm$ 4.2	62.3 $\pm$ 4.3	45.3 $\pm$ 10.9	41.7 $\pm$ 6.6	64.0 $\pm$ 16.7
	+Judge	89.1 $\pm$ 5.6	84.0 $\pm$ 4.7	83.4 $\pm$ 8.1	92.6 $\pm$ 5.5	96.6 $\pm$ 3.5	92.2 $\pm$ 3.0
	+Cluster+Judge	75.6 $\pm$ 7.7	79.4 $\pm$ 7.0	72.6 $\pm$ 9.4	70.1 $\pm$ 7.2	60.4 $\pm$ 8.1	68.4 $\pm$ 7.5
P2	Base	90.9 $\pm$ 1.3	87.4 $\pm$ 16.3	93.3 $\pm$ 2.4	94.3 $\pm$ 5.1	98.9 $\pm$ 2.5	98.1 $\pm$ 2.7
	+Cluster	77.5 $\pm$ 7.3	88.8 $\pm$ 7.1	81.1 $\pm$ 4.5	51.8 $\pm$ 2.9	52.6 $\pm$ 5.0	78.9 $\pm$ 6.6
	+Judge	94.1 $\pm$ 3.7	93.9 $\pm$ 3.8	95.2 $\pm$ 3.4	97.0 $\pm$ 3.9	99.2 $\pm$ 1.8	98.9 $\pm$ 2.1
	+Cluster+Judge	86.3 $\pm$ 7.6	86.5 $\pm$ 8.2	84.6 $\pm$ 8.5	76.8 $\pm$ 7.4	69.3 $\pm$ 9.5	78.5 $\pm$ 8.8
P3	Base	72.6 $\pm$ 2.8	73.3 $\pm$ 3.6	74.9 $\pm$ 4.2	88.4 $\pm$ 2.8	86.9 $\pm$ 5.7	92.4 $\pm$ 2.7
	+Cluster	76.2 $\pm$ 9.8	75.2 $\pm$ 7.4	75.2 $\pm$ 3.6	52.8 $\pm$ 3.9	61.1 $\pm$ 6.4	76.8 $\pm$ 7.1
	+Judge	72.2 $\pm$ 1.8	73.3 $\pm$ 3.1	77.3 $\pm$ 4.8	89.7 $\pm$ 1.8	89.0 $\pm$ 5.1	93.3 $\pm$ 2.7
	+Cluster+Judge	79.2 $\pm$ 5.6	78.9 $\pm$ 6.5	77.7 $\pm$ 4.5	73.3 $\pm$ 7.8	77.3 $\pm$ 9.6	78.7 $\pm$ 7.4
P4	Base	85.7 $\pm$ 0.0	85.7 $\pm$ 0.0	88.4 $\pm$ 3.1	91.0 $\pm$ 3.7	89.3 $\pm$ 5.9	91.0 $\pm$ 4.0
	+Cluster	76.2 $\pm$ 6.6	85.7 $\pm$ 5.7	83.0 $\pm$ 4.8	58.1 $\pm$ 19.0	76.8 $\pm$ 8.9	72.6 $\pm$ 14.0
	+Judge	92.2 $\pm$ 5.1	94.7 $\pm$ 2.5	93.1 $\pm$ 3.4	89.9 $\pm$ 4.0	91.6 $\pm$ 5.5	91.4 $\pm$ 4.3
	+Cluster+Judge	81.3 $\pm$ 5.3	86.5 $\pm$ 5.9	89.9 $\pm$ 5.6	73.5 $\pm$ 12.8	84.4 $\pm$ 5.9	75.4 $\pm$ 10.1
P5	Base	74.3 $\pm$ 3.6	71.0 $\pm$ 6.1	77.3 $\pm$ 6.3	74.9 $\pm$ 2.9	77.3 $\pm$ 4.6	75.8 $\pm$ 2.7
	+Cluster	76.8 $\pm$ 4.0	73.0 $\pm$ 3.8	77.0 $\pm$ 3.6	53.3 $\pm$ 6.7	45.3 $\pm$ 7.7	76.2 $\pm$ 9.3
	+Judge	74.5 $\pm$ 3.3	73.5 $\pm$ 5.8	77.9 $\pm$ 4.7	74.9 $\pm$ 3.8	77.1 $\pm$ 3.9	75.8 $\pm$ 2.3
	+Cluster+Judge	74.9 $\pm$ 7.3	73.5 $\pm$ 6.2	76.2 $\pm$ 5.0	65.5 $\pm$ 12.3	66.7 $\pm$ 13.6	71.0 $\pm$ 8.1
Best E_{Acc} configuration		+Judge	+Judge	+Judge	+Judge	+Judge	+Judge

RQ3.3 Summary

Our repair loop allows 20B parameter models to achieve OML generation success rates comparable to a 120B model, having up to a **39.6%** improvement. Practitioners using medium-sized models should plan for three to four generation attempts to reach optimal reliability, while 120B models see diminishing returns beyond two retries. Researchers should also account for input data complexity when benchmarking model generation with LLMs.

Overall Recommendations

Based on our evaluation of local LLM extraction and OML generation, we recommend the following configurations depending on hardware and budget constraints:

- Small models ($<14\text{B}$ parameters) are sufficient for data extraction but reliably fail at OML generation. Medium-sized models ($\sim 20\text{B}$) perform both extraction and generation effectively when paired with an ontology repair loop.
- Implement at least two rounds of reasoner-based refinement for OML generation using medium-sized models.
- Implement a judge loop only if computational budget and time permit, as the modest accuracy gains ($\sim 2\%$) come at a high latency cost. Avoid clustering mechanisms for semantic extraction tasks, as they fracture context and actively degrade performance for larger models.

Table 6: Performance comparison on OML generation ($S_1(\%)$ / $R_4(\%)$, $n = 25$). S_1 denotes the first round success rate, while R_4 denotes the cumulative success rate after four retries. **Blue** denotes best. Papers IDs defined in Section 4.1.

Paper ID	Configuration	ministral-3 (3B)	ministral-3 (8B)	ministral-3 (14B)	qwen3-next (80B)	gpt-oss (20B)	gpt-oss (120B)
P1	Base	4 / 12	0 / 4	4 / 24	12 / 76	28 / 84	60 / 84
	+Cluster	12 / 16	0 / 24	32 / 72	64 / 72	88 / 96	92 / 92
	+Judge	0 / 0	0 / 4	4 / 24	24 / 68	48 / 88	56 / 92
	+Cluster+Judge	0 / 4	4 / 24	20 / 56	56 / 76	72 / 92	76 / 92
P2	Base	0 / 0	4 / 4	4 / 32	4 / 52	40 / 92	64 / 76
	+Cluster	0 / 0	4 / 28	20 / 52	20 / 60	56 / 96	72 / 96
	+Judge	0 / 0	0 / 4	0 / 20	4 / 64	32 / 68	48 / 80
	+Cluster+Judge	0 / 4	0 / 4	16 / 32	16 / 44	40 / 88	56 / 76
P3	Base	0 / 0	0 / 0	0 / 16	24 / 64	44 / 92	56 / 64
	+Cluster	0 / 8	4 / 32	12 / 64	52 / 72	80 / 96	64 / 80
	+Judge	0 / 0	0 / 4	0 / 20	8 / 44	36 / 100	84 / 96
	+Cluster+Judge	0 / 0	0 / 16	8 / 48	28 / 56	60 / 92	44 / 68
P4	Base	0 / 0	0 / 12	8 / 44	36 / 72	44 / 96	44 / 60
	+Cluster	0 / 0	0 / 8	8 / 64	32 / 68	60 / 84	56 / 84
	+Judge	0 / 16	0 / 0	20 / 56	36 / 60	60 / 92	48 / 72
	+Cluster+Judge	0 / 12	4 / 20	20 / 44	28 / 52	56 / 96	56 / 68
P5	Base	0 / 8	0 / 4	4 / 28	44 / 72	28 / 80	80 / 96
	+Cluster	0 / 0	0 / 8	8 / 48	80 / 92	68 / 100	56 / 80
	+Judge	4 / 8	0 / 0	0 / 8	32 / 72	28 / 96	64 / 84
	+Cluster+Judge	0 / 20	0 / 24	0 / 12	40 / 72	56 / 88	68 / 84
Avg. Improv. ($R_4 - S_1$)		4.4%	10.2%	28.8%	33.4%	39.6%	19.0%

7 Discussion

This section discusses our approach, including the modeling pain points we address, lessons learned from preliminary experimentation, and our threats to validity.

7.1 Addressing Pain Points

Prior studies [64, 65] identified 71 pain points associated with domain modeling, and synthesized the 16 most significant and frequently encountered ones. By employing local LLMs to automatically generate accurate DT reports aligning with the DTDF ontology, DTscribe attempts to address several of those pain points:

(PP04) *Effortful Negotiation*: This pain point refers to the time-consuming process of reaching agreement among domain experts. The automatically generated DT characteristics table produced by our LLM-based system provides initial concept definitions, serving as a shared foundation

upon which experts can iterate, thereby reducing negotiation effort.

(PP07) *Unclear Conceptualization*: Writing clear and complete model definitions is often considered difficult and tedious. In DTInsight [7], we simplified conceptualization by formalizing the DTDF in OML, which acts as a *domain-specific language* for ontology definition. In this paper, we extend that solution by automatically generating an initial OML description that the user can then edit to streamline modeling and reduce cognitive load.

(PP11) *Effortful Diagramming*: Creating comprehensible visual representations of ontologies can be labor-intensive. By generating an ontology compatible with DTInsight, our approach produces clear and structured visualizations automatically. This eases the burden of manual diagramming.

(PP16) *Unclear Documentation*: Documentation in DT modeling is often sparse or ambiguous. The automatic generation of ontologies, especially when integrated with DTInsight, enables clearer,

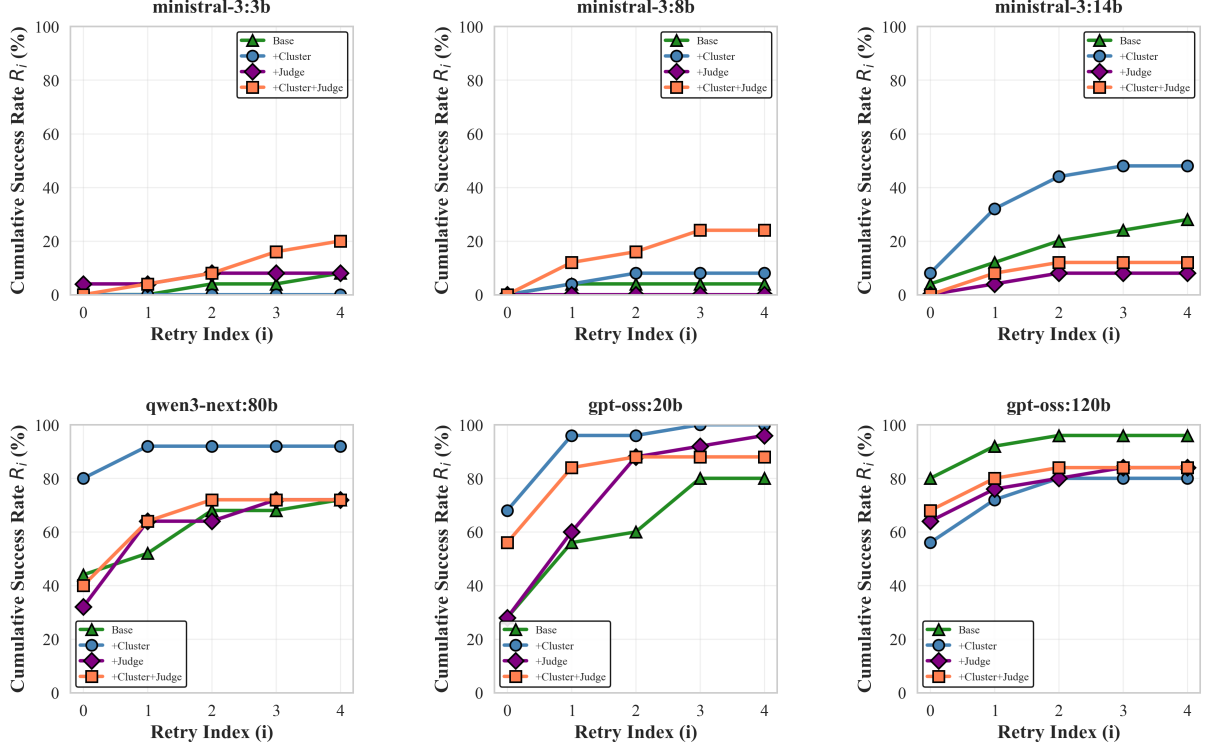


Fig. 7: Cumulative success rate of OML generation across four repair attempts ($n = 25$, P_5).

more interactive, and continuously updated documentation.

Previous research has suggested that “the additional time required for certain modeling tasks, such as negotiating concept definitions and generating verbalizations, [is] necessary rather than wasteful” [64], and thus automating reporting may prove counterproductive as definitions may not have been properly accepted by all parties. However, we still believe having content to discuss rather than starting from scratch helps modelers and experts reach agreement faster [15].

7.2 Lessons Learned

This section details some lessons learned during our preliminary experiments and potential future approach improvements.

7.2.1 Multimodal Document Pre-processing

We experimented with local multimodal pre-processing with a vision LLM (`qwen2.5-vl-7b`) to simultaneously extract text and generate textual descriptions for figures. However, we discarded this method due to significant limitations: (a) reliability issues, as the LLM frequently hallucinated content or omitted sentences; (b) high latency, requiring approximately one minute per page; (c) instability, characterized by occasional infinite loops; and (d) system complexity, which introduced an unnecessary point of failure. While general-purpose vision LLMs proved unsuitable, specialized multimodal parsing tools such as Docling¹⁹ may offer a more robust solution.

7.2.2 Hyperparameter Tuning

Despite the literature suggesting hyperparameter tuning improves domain modeling with LLMs [66],

¹⁹<https://www.docling.ai/>

our preliminary experiments did not generate substantial benefits. As such, we do not utilize finetuning in our approach, but instead highlight that there may be potential in improving LLM output quality further.

7.2.3 Local/Hosted Approach Tradeoff

A local deployment setup on consumer GPU with 8 GB of VRAM enabled characteristic extraction and OML generation to be performed directly within our pipeline, providing data privacy and full control over parameters and input data. However, we observed occasional slowdowns under repeated retry loops and large context windows proportional to those shown in Table 4. Preliminary experiments showed certain smaller models, such as llama3:8B, showed inconsistencies when forced to comply with structured outputs.

In contrast, cloud-based services such as Claude²⁰ or ChatGPT²¹ provide high-quality outputs even from unstructured inputs, due to the robustness of their proprietary architectures and larger model size. This reduces the need for aggressive pre-processing and strict prompting strategies. Additionally, external API services such as Ollama Cloud provide lower latency and access to bigger models, reducing the computational burden. However, relying on cloud services introduces limitations related to data privacy, reproducibility risks due to provider-side updates or pricing changes, and availability constraints linked to network connectivity and provider reliability.

In practice, the deployment depends on operational constraints. When sufficient hardware is available and continuous integration is required, a locally deployed setup using API-accessible models can balance performance and control. In contrast, when privacy requirements are minimal and hardware capacity is limited, a fully cloud-based solution may offer higher baseline performance. As open-source models continue to improve, the performance gap between local and hosted approaches is expected to narrow, potentially favoring fully local deployments in future industrial settings.

²⁰<https://claude.ai/>

²¹<https://chatgpt.com/>

7.3 Threats to Validity

Here we present the threats to validity of our article, based on Wohlin *et al.* [67].

7.3.1 Internal Validity

Threats to internal validity refer to possible errors and biases in our experimentation.

Despite its effectiveness, LLM-based evaluation presents limitations such as lack of reliability and accuracy. Reliability issues arise from the inherent stochastic nature of LLMs. To mitigate this, we enforce deterministic settings by fixing the random seed and setting the temperature to zero to improve consistency across runs.

However, despite deterministic configurations, LLM-based judges may still be inaccurate, producing stable yet misaligned evaluations with respect to human expert judgment. Because of this, while the LLM provides a scalable and consistent evaluation mechanism, its assessments should not be considered equivalent to those of a human domain expert, but rather as an automated proxy to support iterative refinement.

The output quality of LLMs also heavily relies on prompts [54] and architecture. To mitigate the effects of these prompts, we follow the guidelines of Baltes *et al.* [61] and report LLM versioning and the main prompt used in Listing 2. Similarly, retrieval performance also relies on the queries used, as very detailed and complex queries return low semantically similar fragments. Therefore, we chose to use associated keywords, keeping these as broad as possible while still preserving the meaning of each cluster.

For the definition of a partial ground truth (Section 4.2), we acknowledge the threats to validity presented by the subjective human evaluation. However, this is mitigated by having three independent raters annotate the characteristics, and our resulting Fleiss' Kappa score of $\kappa = 0.625$. This value indicates *substantial agreement* [59, 60], indicating that our ratings are consistent.

Internal validity is also threatened when we use cloud-hosted LLMs, as the throughput can vary over time due to shared resources and provider-side changes. However, we considered that a cloud-based platform was more stable and performant than running our large and multi-run experimentation on a laptop machine.

7.3.2 Construct Validity

Construct validity addresses the choice of evaluation metrics.

From the literature, suitable metrics for our precise problem definition were not found. Thus, we defined *RQ2* to determine these metrics, as discussed in Section 4. These include LLM-judging and comparison against a partial human-defined ground truth, to provide qualitative and quantitative results of the output. Note that a partial ground truth was defined in this work as we lack a robust mapping of DTs and their descriptions in the DTDF. Thus we were unable to use standard metrics such as recall and accuracy.

Another limitation is our focus on OML validity from the reasoner as a measure of success. In some rare cases, we noticed the LLM could “cheat” by generating over-simplified/partial OML to pass this validation. This usually happened after three OML refinement loops, suggesting the LLM can get stuck repairing the ontology and fails to generate a complete OML description.

As mentioned earlier, a lower characteristic extraction accuracy also tends to produce a higher OML generation success rate, as fewer model elements are produced. This metric therefore somewhat depends on the extraction accuracy and cannot be measured independently. Thus, we acknowledge that this OML generation metric has issues, but it does produce a quantifiable result for pipeline integration and analysis.

7.3.3 Conclusion Validity

Conclusion validity concerns whether the conclusions we draw are supported by the results.

Despite the stochastic nature of LLMs, we are confident in the results produced by our rigorous experimentation. Based on these results, we have drawn solid insights and recommendations on the application of LLMs to characteristic extraction and model generation.

7.3.4 External Validity

The threat of external validity relates to the generalizability of our conclusions.

The time- and compute-intensive nature of our experimentation necessarily limited the number of papers, LLMs, and configurations that we could investigate.

For our experimentation, we chose six LLMs with a variety of parameter sizes and model families. We believe that this mitigates the threat of using other LLMs in the approach. We also note that LLM performance is increasing, so future models are likely to perform better. For the configurations, we intended to investigate the effects of clustering and LLM-as-a-judge, which are the major decisions of our approach.

For the paper selection, we did use *convenience sampling* by choosing papers from the 2025 conference on Engineering Digital Twins. While this is a limited sample, our intention was to determine the effectiveness of characteristic extraction from high-quality DT exemplars which do not explicitly align with the DT description framework.

Jin *et al.* [8] note that internal company documents tend to be more complex than the cases used in this study. LLMs may struggle to effectively extract information from extensive and highly complex documents, particularly when data is distributed across many sources. Thus, our results may provide an upper bound on the extraction quality compared to use in practice. However, our core contributions of the approach and insights still provide a baseline approach and evaluation for further development.

8 Related Work

This section reports on related work concerning LLMs as applied to DTs and model/ontology generation.

8.1 LLMs for DTs

There is some initial work on applying LLMs to DTs, primarily to reduce the technical barrier in understanding and integrating technical formats. As far as we are aware, we are the first to target the DT reporting challenge.

Giabbanelli *et al.* suggested using LLMs for integrating modeling and simulation tools [34, 35]. They regard ontologies as potential candidates to formally specify concepts to be shared among models, thus mitigating LLM hallucinations. They also consider employing LLMs “for generating human-readable explanations of source code, simulation models, and simulation results at various levels of technical depth” [34]. However, our goal

is *conceptual* DT characteristic report generation. In particular we target a pre-established and OML-formalized DTDF, to facilitate high-level communication and planning among stakeholders.

Gebreab *et al.* [33] investigated the use of generative AI for data integration in DTs using ontologies, as well as for generating 3D models. Koskinen *et al.* [68] investigated the use of LLMs for generating DT simulation code. Our work instead focuses on DT reporting with LLMs rather than actual construction and integration of DT components.

Other approaches have also considered using LLMs to provide natural language interaction with DTs. In particular, Zhang *et al.* [69] provide a framework for utilizing LLMs to a) explain the DT’s decisions, b) transform the user’s natural language commands into precise DT actions, and c) with the LLM as a planner itself. Kirner *et al.* [70] use a locally-hosted LLM, a RAG approach, and a chat-based UI to allow the user to easily query information about a construction site. Again, while these approaches share techniques, we address the problem of extracting and presenting concrete DT characteristics from text reports.

8.2 LLMs for Domain Modeling

The literature has also investigated the application of LLMs to model and ontology generation.

8.2.1 Guiding LLMs

Despite promising success in generation tasks, the well-known issues of semantic misalignment and hallucinations means that LLMs are not perfect and require guidance in their generation [71].

For example, Pan *et al.* [19] utilize an Ecore meta-model to guide the LLM to produce valid JSON files, which are then transformed into XMI format. Bozyigit *et al.* [72] investigated automated conceptual model generation from requirements, comparing an LLM with a grammatical rule-based technique. Qualis examined repairing SysML v2 generation errors using LLMs, and argued that future work could focus on “ontology-backed verification [to] improve efficiency, reproducibility, and the reliability of benchmarking outcomes” [63]. We in fact explicitly use OML with its integrated semantic reasoner, using it to

benchmark the improvements of multiple rounds of LLM-based but reasoner-backed repairs.

Chen *et al.* focused on *fully automated* domain modeling without any human intervention using LLMs [10]. They evaluate the generated results manually with a consensus approach, comparing it to the reference solution of ten problem descriptions from an undergraduate course. Chen *et al.* later developed a framework for improving the consistency and quality of LLM-generated graphs using the Mermaid diagramming language. It synthesizes candidate graphs to generate the optimal one. It is particularly efficient the smaller the LLM, with five candidates needed for optimal quality improvement using Llama3.1-8B [73]. In contrast, our focus is on fully automated *DT characteristics extraction* that we extended to domain modeling using ontologies to allow the modeler to visualize the result as reported by DTInsight.

Mehenni and Zouaq [74] propose an approach close to ours, where they extract medical concepts and attributes from clinical notes. They use ontology-augmented LLM prompts with a custom constrained decoding strategy, such that the LLM produces summaries using concepts described in the ontology. In contrast, our approach enforces that the OML description model produced by the LLM must conform to the OML vocabulary, and directs the LLM to fix any detected non-conformity.

8.2.2 Requirements and Architecture Models

Beyond system models, LLMs have also been used for requirement specification and architecture description. For example, Jin *et al.* [8] performed requirements modeling for CPSs using LLMs to generate a *problem diagram*. Hatahet *et al.* [75] generate software architecture diagrams and state machines by providing an LLM with a class diagram and associated code.

We instead focus on DT reporting, by targeting DT characteristics modeling with ontologies to represent a *conceptual architecture diagram* (DT constellation) and plain-text characteristics.

8.2.3 Ontologies and Knowledge Graphs

In the literature, research on LLM-generated ontologies has focused on OWL generation [13, 15, 16] rather than OML [40] generation. The exception is two unpublished works from NASA JPL. OML Code²² is a VSCode extension being developed for supporting OML generation with LLMs, with a graphical viewer and editor. Another project²³ is similar to our work, where documents are analyzed based on an OML vocabulary. However, these works are unpublished and do not tackle the precise case of DT reporting that we do.

Yang *et al.* leverage LLMs for ontology and knowledge graph construction in the medical domain [76]. They use a hand-built ontology and LLM-generated competency questions (CQs) to expand the knowledge graph, using RAG on PubMed literature data. An LLM judge evaluates responses against the ground truth from a one to five scale using four criteria: *accuracy*, *completeness*, *relevance*, and *consistency*. Kommineni *et al.* [77] take a very similar approach, and applies this approach to deep learning methodologies. In our case, rather than using an ontology to scope and structure the LLM output, the DTDF vocabulary guides DTDF characteristics extraction, later used for OML description generation.

Lippolis *et al.* [15] generated OWL ontology drafts using LLMs based on ontology requirements such as user stories and competency questions (CQs). They proposed two methods, independent and incremental ontology generation, evaluated with OOPS! [78] and qualitatively assessed by domain experts. Their work builds on that of Saeedizade and Blomqvist [13] who find that smaller LLMs are not able to consistently generate OWL without requiring fine-tuning and extensive prompt engineering, as in [17, 79]. In contrast, our approach employs LLMs to generate complete OML ontologies directly from documents and iteratively refine them using reasoner feedback.

Chiş *et al.* [80] developed a “low-code knowledge graph building” tool for easily creating ontologies from a visual drag-and-drop interface, and thus democratize the use of knowledge graphs.

Similar to our proposal, they avoid RDF/OWL jargon and encourage users to think about component relationships. However, they do not target DTs nor use OML, and instead propose a general solution for representing flexible graph structures. However, this makes KG creation complex, and does not align with the DTDF.

Chat interfaces for building ontologies with LLMs have also been proposed. Zhang *et al.* [16] employed chat-based LLMs to generate CQs from a domain-expert user story, which were used by ontology engineers to model the ontology. In contrast, we use pre-existing documents to fully automate DT reporting in the DTDF ontology. Similarly, Coutinho *et al.* [81] provide an LLM assistant to generate ontologies based on UFO. In contrast, our work does not have the user in the loop, other than to indicate the source documents and potentially correct the outputted DT description model.

9 Conclusion

This article has described our DTscribe approach to automate the reporting of Digital Twins (DTs). Our approach ingests written descriptions of DTs, and then extracts (using local LLMs) textual and structure characteristics based on our Digital Twin Description Framework (DTDF) [4]. LLMs are also used to generate *description models* in the Ontological Modeling Language (OML) to formally represent the DT. These models are queried by our previously-developed DTInsight tool [7] to produce an interactive visualization and reporting page for the DT²⁴.

From our evaluation and results, we show that it is feasible to automate this process with local LLMs. We define quality metrics for our approach including extraction accuracy and generation success, and correlate the decisions in our approach decisions with increases or decreases in these qualities. We then provide specific recommendations on this topic, such as utilizing medium sized models, employing two or three model refinement loops, and avoiding an LLM-as-a-judge or clustering approach.

²²<https://github.com/opencaesar/oml-code>

²³<https://www.opencaesar.io/projects/2024-04-12-Document-Analyzer-Using-LLM.html>

²⁴Such as <https://oakeslabmtl.github.io/dtdf-dtscribe/>

Future work

On the technical side, there are further directions to improve the capabilities and quality of DTScribe.

For example, one of our next steps is to expand the input data for DTScribe to include multi-modal local or hosted documents from diverse sources. This includes videos, audio recordings, meeting notes, images, etc. which may contain valuable information about the DT.

Another possible improvement is to investigate fine-tuning of the LLMs, as recommended by [17]. In particular, using different fine-tuned LLMs for characteristic extraction and OML ontology generation may improve quality and reliability.

The framework itself could use further detailing, with SHERPA [82] or Graph of Thoughts (GoT) [83] employed to improve LLM prompting, and automatically search existing knowledge to build the ontology.

One aspect of our current approach is that we do not customize the produced report to individual stakeholders. While this is technically feasible through prompt engineering, we decided to not pursue this customization. Instead, the reports are generated as a general representation since the DT constellation and the reports are intended to support a shared understanding across stakeholders.

Future work could define quality metrics and model generation based on predefined personas [84]. The goal is to address the “Effortful Verbalization” [64, 65] pain point (PP05) by automatically generating DT reporting pages that meet the user at their level of expertise, tailored to different project stakeholders such as a manager or an engineer. We could enable weighting of the final output to generate more detailed or broader reports. This would resemble the collection of ontology requirements from user stories [16], but here we would collect DT characteristics from documents to generate an *ontology-based DT report*.

The user could also be involved to co-construct the DTDF ontology with an LLM agent. Here, the agent could a) create competency questions to evaluate the knowledge it has formulated with regards to the documents it has, or b) directly ask the user for details when missing information about a characteristic, and then update the DT model over time to ensure alignment with the

actual DT. This resembles past work to help ontology engineers model ontologies [16], and other recent works [85–88].

We are also in discussions with larger research networks to develop DT descriptions for their DTs. We plan to conduct experiments to build DT reporting pages for their DTs, and validate our approach and tooling at a wider scale.

10 Acknowledgments

We would like to thank Boqi (Percy) Chen for his feedback and insights for domain modeling with LLMs. This research was supported by a MITACS Globalink Research Internship and the NSERC Discovery project *Accelerating DT Construction* (no. RGPIN-2024-05622).

References

- [1] Madni, A.M., Madni, C.C., Lucero, S.D.: Leveraging digital twin technology in model-based systems engineering. *Systems* **7**(1), 7 (2019)
- [2] Combemale, B., Vicat-Blanc, P., Blouin, A., El Haouzi, H.B., Bruel, J.-M., Deantoni, J., Duval, T., Gérard, S., Jézéquel, J.-M.: Engineering digital twins: A research roadmap. In: EDTconf 2025-2nd International Conference on Engineering Digital Twins (2025)
- [3] Alskaf, T., Babur, Ö., Bordeleau, F., Cleophas, L., Combemale, B., Denil, J., Haugen, Ø., Michael, J., Nguyen, P., Secoleanu, T., *et al.*: Evolution at the core of digital twin engineering. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 210–216 (2025). IEEE
- [4] Gil, S., Oakes, B., Gomes, C., Frasheri, M., Larsen, P.G.: Toward a systematic reporting framework for Digital Twins: A cooperative robotics case study. *SIMULATION* **101**(3), 313–339 (2024) <https://doi.org/10.1177/00375497241261406>
- [5] Oakes, B., Parsai, A., Van Mierlo, S., Demeyer, S., Denil, J., De Meulenaere,

- P., Vangheluwe, H.: Improving digital twin experience reports. In: Proceedings of the 9th International Conference on Model-Driven Engineering and Software Development (MODELSWARD), pp. 179–190. SciTePress, Vienna, Austria (2021). <https://doi.org/10.5220/0010236101790190>. INSTICC
- [6] Goffi, P.-E., Tremblay, R., Oakes, B.: Engineering a digital twin for the monitoring and control of beer fermentation sampling. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 167–173 (2025). IEEE
- [7] Fiter, K., Malassigné-Onfroy, L., Oakes, B.: Dtinsight: A tool for explicit, interactive, and continuous digital twin reporting. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 139–143 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00030>
- [8] Jin, D., Zhao, S., Jin, Z., Chen, X., Wang, C., Fang, Z., Xiao, H.: An evaluation of requirements modeling for cyber-physical systems via LLMs. arXiv preprint arXiv:2408.02450 (2024)
- [9] Burgueño, L., Di Ruscio, D., Sahraoui, H., Wimmer, M.: Automation in model-driven engineering: A look back, and ahead. *ACM Transactions on Software Engineering and Methodology* **34**(5), 1–25 (2025)
- [10] Chen, K., Yang, Y., Chen, B., Hernández López, J.A., Mussbacher, G., Varró, D.: Automated Domain Modeling with Large Language Models: A Comparative Study. In: 2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS), pp. 162–172 (2023). <https://doi.org/10.1109/MODELS58315.2023.00037>
- [11] Noy, N.F., Hafner, C.D.: The state of the art in ontology design: A survey and comparative review. *AI magazine* **18**(3), 53–53 (1997)
- [12] Du, R., An, H., Wang, K., Liu, W.: A short review for ontology learning: Stride to large language models trend. arXiv preprint arXiv:2404.14991 (2024)
- [13] Saeedizade, M.J., Blomqvist, E.: Navigating ontology development with large language models. In: European Semantic Web Conference, pp. 143–161 (2024). Springer
- [14] Fathallah, N., Das, A., Giorgis, S.D., Poltronieri, A., Haase, P., Kovriguina, L.: Neon-gpt: a large language model-powered pipeline for ontology learning. In: European Semantic Web Conference, pp. 36–50 (2024). Springer
- [15] Lippolis, A.S., Saeedizade, M.J., Keskiärrkkä, R., Zuppiroli, S., Ceriani, M., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Ontology generation using large language models. In: European Semantic Web Conference, pp. 321–341 (2025). Springer
- [16] Zhang, B., Carriero, V.A., Schreiberhuber, K., Tsaneva, S., González, L.S., Kim, J., Berardinis, J.: Ontochat: a framework for conversational ontology engineering using language models. In: European Semantic Web Conference, pp. 102–121 (2024). Springer
- [17] Babaei Giglou, H., D’Souza, J., Auer, S.: LLMs4OL: Large language models for ontology learning. In: International Semantic Web Conference, pp. 408–427 (2023). Springer
- [18] Shimizu, C., Hitzler, P.: Accelerating knowledge graph and ontology engineering with large language models. *Journal of Web Semantics* **85**, 100862 (2025)
- [19] Pan, F., Petrovic, N., Zolfaghari, V., Wen, L., Knoll, A.: Llm-enabled instance model generation. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 586–595 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00082>
- [20] Gil, S., Bas, E.E., Jensen, C.D., Engelsgaard,

- S., Abbiati, G., Gomes, C.: Fmi-based distributed co-simulation with enhanced security and intellectual property safeguards. In: Proceedings of the 2025 Annual Modeling and Simulation Conference (ANNSIM'25) (2025)
- [21] Jacoby, M., Volz, F., Weißenbacher, C., Stojanovic, L., Usländer, T.: An approach for industrie 4.0-compliant and data-sovereign digital twins: Realization of the industrie 4.0 asset administration shell with a data-sovereignty extension. *at-Automatisierungstechnik* **69**(12), 1051–1061 (2021)
- [22] Imanifard, A., Majidi, B., Shamisa, A.: Smartgridagent: An educational framework for reliable digital twin-based smart grid workforce training with locally hosted LLMs. *Smart Grids and Sustainable Energy* **10**(2), 41 (2025)
- [23] Somma, A., Amalfitano, D., De Benedictis, A., Pelliccione, P.: TwinArch: A digital twin reference architecture. *Journal of Systems and Software* **231**, 112613 (2026) <https://doi.org/10.1016/j.jss.2025.112613>
- [24] Hugues, J., Hristosov, A., Hudak, J.J., Yankel, J.: TwinOps - DevOps meets model-based engineering and digital twins for the engineering of CPS. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, pp. 1–5. ACM, Virtual Event Canada (2020). <https://doi.org/10.1145/3417990.3421446>
- [25] Ramdhan, S., Trandinh, W., David, I., Pantelic, V., Lawford, M.: Engineering automotive digital twins on standardized architectures: A case study. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 174–180 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00034>
- [26] Michael, J., Cleophas, L., Zschaler, S., Clark, T., Combemale, B., Godfrey, T., Khelladi, D.E., Kulkarni, V., Lehner, D., Rumpe, B., Wimmer, M., Wortmann, A., Ali, S., Barn, B., Barosan, I., Bencomo, N., Bordeleau, F., Grossmann, G., Karsai, G., Kopp, O., Mitschang, B., Muñoz Ariza, P., Pierantonio, A., Polack, F.A.C., Riebisch, M., Schlingloff, H., Stumptner, M., Vallecillo, A., Van Den Brand, M., Vangheluwe, H.: Model-Driven Engineering for Digital Twins: Opportunities and Challenges. *Systems Engineering*, 21815 (2025) <https://doi.org/10.1002/sys.21815>
- [27] Liebel, G., Klünder, J., Hebig, R., Lazik, C., Nunes, I., Graßl, I., Steghöfer, J.-P., Exelmans, J., Oertel, J., Marquardt, K., *et al.*: Human factors in model-driven engineering: future research goals and initiatives for mde. *Software and Systems Modeling* **23**(4), 801–819 (2024)
- [28] Karabulut, E., Pileggi, S.F., Groth, P., Degeler, V.: Ontologies in digital twins: A systematic literature review. *Future Generation Computer Systems* **153**, 442–456 (2024)
- [29] Zheng, X., Lu, J., Kiritsis, D.: The emergence of cognitive digital twin: vision, challenges and opportunities. *International Journal of Production Research* **60**(24), 7610–7632 (2022)
- [30] Oakes, B., Gomes, C., Kamburjan, E., Abbiati, G., Ecem Bas, E., Engelsgaard, S.: Towards ontological service-driven engineering of digital twins. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, pp. 464–469 (2024)
- [31] Lu, C., Valle, P., Wu, J., Isaku, E., Sartaj, H., Arrieta, A., Ali, S.: Foundation Models for Software Engineering of Cyber-Physical Systems: The Road Ahead. *arXiv* (2025). <https://doi.org/10.48550/arXiv.2504.04630>
- [32] Li, M., Wang, R., Zhou, X., Zhu, Z., Wen, Y., Tan, R.: ChatTwin: Toward Automated Digital Twin Generation for Data Center via Large Language Models. In: Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, pp. 208–211.

- ACM, Istanbul Turkey (2023). <https://doi.org/10.1145/3600100.3623719>
- [33] Gebreab, S., Musamih, A., Salah, K., Jayaraman, R., Bosovic, D.: Accelerating digital twin development with generative ai: A framework for 3d modeling and data integration. *IEEE Access* **12**, 185918–185936 (2024) <https://doi.org/10.1109/ACCESS.2024.3514175>
 - [34] Giabbanelli, P.J., Beverley, J., David, I., Tolk, A.: From over-reliance to smart integration: using large-language models as translators between specialized modeling and simulation tools. *arXiv preprint arXiv:2506.11141* (2025)
 - [35] Giabbanelli, P.J.: A Guide to Large Language Models in Modeling and Simulation: From Core Techniques to Critical Challenges (2026). <https://arxiv.org/abs/2602.05883>
 - [36] Cheng, H., Husen, J.H., Lu, Y., Racharak, T., Yoshioka, N., Ubayashi, N., Washizaki, H.: Generative AI for requirements engineering: A systematic literature review. *Software: Practice and Experience* (2025)
 - [37] Marcus, G.: The next decade in ai: four steps towards robust artificial intelligence. *arXiv preprint arXiv:2002.06177* (2020)
 - [38] Heithoff, M., Michael, J., Rumpe, B., Pfeiffer, J., Wortmann, A., Zhang, J.: A method for model-driven engineering of digital twins in manufacturing (2025)
 - [39] Escott, E.: Jidoka: Automation with a human touch. *Software and Systems Modeling* **24**(2), 339–358 (2025) <https://doi.org/10.1007/s10270-024-01256-z>
 - [40] Elaasar, M., Rouquette, N., Wagner, D., Oakes, B., Hamou-Lhadj, A., Hamdaqa, M.: openCAESAR: Balancing Agility and Rigor in Model-Based Systems Engineering. In: *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 221–230. IEEE, Västerås, Sweden (2023). <https://doi.org/10.1109/MODELS-C59198>
 - [41] Feng, H., Gomes, C., Thule, C., Lausdahl, K., Sandberg, M., Larsen, P.G.: The incubator case study for digital twin engineering. *arXiv preprint arXiv:2102.10390* (2021)
 - [42] Gruber, T.R.: A translation approach to portable ontology specifications. *Knowledge acquisition* **5**(2), 199–220 (1993)
 - [43] Humphries, L., Bullegas, G., Golledge, J., Mccausland, C., Taylor, D., Kolovos, D., Garcia-Dominguez, A., Gerasimou, S.: Integrated knowledge centric engineering: Delivering next-generation aircraft projects at pace. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 834–843 (2025). IEEE
 - [44] Wu, H., Monahan, R., Power, J.F.: Metamodel instance generation: A systematic literature review. *arXiv preprint arXiv:1211.6322* (2012)
 - [45] Patil, R., Gudivada, V.: A review of current trends, techniques, and challenges in large language models (LLMs). *Applied Sciences* **14**(5), 2074–2074 (2024) <https://doi.org/10.3390/app14052074>
 - [46] Kumar, P.: Large language models (LLMs): Survey, technical frameworks, and future challenges. *Artificial Intelligence Review* **57**, 260 (2024) <https://doi.org/10.1007/s10462-024-10888-y>. Accepted: 25 July 2024; Published: 18 August 2024
 - [47] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., Wang, H.: Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* **2**(1) (2023)
 - [48] Zheng, H., Wang, S., Huang, L.: A survey of document-level information extraction. *arXiv preprint arXiv:2309.13249* (2023)
 - [49] Korkankar, P.D., Abranches, A., Bhagat, P., Pawar, J.: Aspect-based summaries from online product reviews: A comparative study

- using various LLMs. In: Lalitha Devi, S., Arora, K. (eds.) Proceedings of the 21st International Conference on Natural Language Processing (ICON), pp. 562–568. NLP Association of India (NLP AI), AU-KBC Research Centre, Chennai, India (2024). <https://aclanthology.org/2024.icon-1.65/>
- [50] Feng, Y., Zhao, S., Li, Y., Xiao, L., Wu, X., Luu, A.T.: Aspect-based summarization with self-aspect retrieval enhanced generation. arXiv preprint arXiv:2504.13054 (2025)
- [51] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., *et al.*: Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35**, 24824–24837 (2022)
- [52] Zelikman, E., Harik, G., Shao, Y., Jayasiri, V., Haber, N., Goodman, N.D.: Quietstar: Language models can teach themselves to think before speaking. arXiv preprint arXiv:2403.09629 (2024)
- [53] Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., Guo, J.: A Survey on LLM-as-a-Judge (2025). <https://arxiv.org/abs/2411.15594>
- [54] Crupi, G., Tufano, R., Velasco, A., Mastropalo, A., Poshyanyk, D., Bavota, G.: On the effectiveness of llm-as-a-judge for code generation and summarization. *IEEE Transactions on Software Engineering* (2025)
- [55] Huyen, C.: *AI Engineering: Building Applications with Foundation Models*. O’Reilly Media, Inc., Sebastopol, CA (2025)
- [56] Heithoff, M., Trinh, M., Michael, J., Rumpe, B., Brecher, C.: A digital shadow for accurate robot motion control: Integrating data with friction models. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 113–124 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00026>
- [57] Mertens, J., Denil, J.: Lab-scale gantry crane digital twin exemplar. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 217–223 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00039>
- [58] Zech, P., Senoner, S., Goldin, E., Zallinger, C., Hammes, S., Michael, J.: Model-driven digital twins for aeeco. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 224–235 (2025). <https://doi.org/10.1109/MODELS-C68889.2025.00040>
- [59] Fleiss, J.L.: Measuring nominal scale agreement among many raters. *Psychological bulletin* **76**(5), 378 (1971)
- [60] Landis, J.R., Koch, G.G.: The measurement of observer agreement for categorical data. *biometrics*, 159–174 (1977)
- [61] Baltes, S., Angermeier, F., Arora, C., Barón, M.M., Chen, C., Böhme, L., Calefato, F., Ernst, N., Falessi, D., Fitzgerald, B., Fucci, D., Kalinowski, M., Lambiase, S., Russo, D., Lungu, M., Prechelt, L., Ralph, P., Treude, C., Wagner, S.: Guidelines for Empirical Studies in Software Engineering involving Large Language Models (2025). <https://arxiv.org/abs/2508.15503>
- [62] Muennighoff, N., Tazi, N., Magne, L., Reimers, N.: Mteb: Massive text embedding benchmark. arXiv preprint arXiv:2210.07316 (2022)
- [63] Qualis, R.A.: Mitigating hallucinations in SysML v2 generation using LLMs and a tri-layered knowledge graph reasoning framework. In: 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 357–366 (2025). IEEE
- [64] Sousa, I.V., Sales, T.P., Guerra, E., Guizzardi, G., *et al.*: Mapping the pain: How modelers experience and respond to common

- domain modeling frustrations. In: Proceedings of the 29th International Conference on Enterprise Design, Operations, and Computing (EDOC 2025). IEEE, Lisbon, Portugal (2025)
- [65] Sousa, I.V., Sales, T.P., Guerra, E., Daneva, M., Guizzardi, R., Bonino da Silva Santos, L.O., Proper, H.A., Guizzardi, G.: Unraveling the pain points of domain modeling. *Information and Software Technology* **183**, 107736 (2025) <https://doi.org/10.1016/j.infsof.2025.107736>
- [66] Bulhakov, V., d’Aloisio, G., Di Sipio, C., Di Marco, A., Di Ruscio, D.: Investigating the role of LLMs hyperparameter tuning and prompt engineering to support domain modeling. *arXiv preprint arXiv:2507.14735* (2025)
- [67] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A.: *Experimentation in Software Engineering*, 2nd edn., p. 274. Springer, Berlin, Heidelberg (2024). <https://doi.org/10.1007/978-3-662-69306-3>
- [68] Koskinen, A.: Challenges and opportunities in digital twin development: leveraging maturity models and large language models for simulation modeling. Master’s thesis, University of Oulu (2024). <https://oulurepo.oulu.fi/handle/10024/53389>
- [69] Zhang, N., Vergara-Marcillo, C., Diamantopoulos, G., Shen, J., Tziritas, N., Bahsoon, R., Theodoropoulos, G.: Large language models for explainable decisions in dynamic digital twins. In: *International Conference on Dynamic Data Driven Applications Systems*, pp. 81–89 (2024). Springer
- [70] Kirner, L.T., Oraskari, J.T., Brell-Cokcan, S.: Developing a RAG-based system for natural language access to linked building data on construction sites. In: Pauwels, P., Poveda-Villalón, M., Terkaj, W. (eds.) *Proceedings of the 13th Linked Data in Architecture and Construction Workshop (LDAC 2025)*, Co-located with the 42nd International CIB W78 Conference on IT in Construction. *CEUR Workshop Proceedings*, vol. 4113, pp. 66–77. Porto, Portugal (2025). <https://doi.org/10.18154/RWTH-2025-10214> . <https://ceur-ws.org/Vol-4113/paper5.pdf>
- [71] Cámara, J., Troya, J., Burgueño, L., Vallecillo, A.: On the assessment of generative ai in modeling tasks: an experience report with chatgpt and uml. *Software and Systems Modeling* **22**(3), 781–793 (2023)
- [72] Bozyigit, F., Bardakci, T., Khalilipour, A., Challenger, M., Ramackers, G., Babur, Ö., Chaudron, M.R.: Generating domain models from natural language text using nlp: a benchmark dataset and experimental comparison of tools. *Software and Systems Modeling* **23**(6), 1493–1511 (2024)
- [73] Chen, B., Wei, O., Zheng, B., Mussbacher, G.: Accurate and consistent graph model generation from text with large language models. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pp. 130–141 (2025). IEEE
- [74] Mehenni, G., Zouaq, A.: *Ontology-Constrained Generation of Domain-Specific Clinical Summaries*. *arXiv* (2024). <https://doi.org/10.48550/arXiv.2411.15666>
- [75] Hatahet, A., Knieke, C., Rausch, A.: Generating software architecture description from source code using reverse engineering and large language model. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 566–575 (2025). IEEE
- [76] Yang, H., Xiao, L., Zhu, R., Liu, Z., Chen, J.: An llm supported approach to ontology and knowledge graph construction. In: *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pp. 5240–5246 (2024). IEEE
- [77] Kommineni, V.K., König-Ries, B., Samuel, S.: Towards the automation of knowledge graph construction using large language models. In: *NLP4KGC@ SEMANTiCS* (2024)

- [78] Poveda-Villalón, M., Gómez-Pérez, A., Suárez-Figueroa, M.C.: Oops!(ontology pitfall scanner!): An on-line tool for ontology evaluation. *International Journal on Semantic Web and Information Systems (IJSWIS)* **10**(2), 7–34 (2014)
- [79] Mai, H.T., Chu, C.X., Paulheim, H.: Do LLMs really adapt to domains? an ontology learning perspective. In: *International Semantic Web Conference*, pp. 126–143 (2024). Springer
- [80] Chiş, A., Ghiran, A.-M., Buchmann, R.A.: GRAPHxx: DSML engineering for knowledge graph building and streamlining with GraphRAG. *Software and Systems Modeling* (2025) <https://doi.org/10.1007/s10270-025-01297-y>
- [81] Coutinho, M.L., Almeida, J.P.A., Guizzardi, G.: LLM-based modeling assistance for textual ontology-driven conceptual modeling. In: *44th International Conference on Conceptual Modeling (ER 2025) Demo Track. CEUR Workshop Proceedings*, pp. 338–342. CEUR-WS.org, Poitiers, France (2025)
- [82] Chen, B., Chene, K., López, J.A.H., Mussbacher, G., Varró, D., Feizpour, A.: Sherpa: A model-driven framework for large language model execution. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pp. 197–208 (2025). IEEE
- [83] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., *et al.*: Graph of thoughts: Solving elaborate problems with large language models. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 17682–17690 (2024)
- [84] Shankar, S., Zamfirescu-Pereira, J., Hartmann, B., Parameswaran, A., Arawjo, I.: Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In: *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–14 (2024)
- [85] Cabot, J.: Vibe modeling: Challenges and opportunities. In: *International Conference on Conceptual Modeling*, pp. 105–118 (2025). Springer
- [86] Silva, J., Ma, Q., Cabot, J., Kelsen, P., Proper, H.A.: Towards human-in-the-loop llm-enabled domain modeling. In: *International Conference on Conceptual Modeling*, pp. 127–145 (2025). Springer
- [87] Burgueño, L., Keet, M., Kienzle, J., Michael, J., Babur, Ö.: A human behavior exploration approach using LLMs for cyber-physical systems. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pp. 578–586 (2024)
- [88] Braun, G., Gimenez, C., Cecchi, L., Fillottrani, P.: crowd: A visual tool for involving stakeholders into ontology engineering tasks. *KI-Künstliche Intelligenz* **34**(3), 365–371 (2020)

A Appendix

Table 7: DTDF [4] characteristic extraction prompts across all clusters.

$\mathcal{CC}_1$: Purpose Cluster	
C_1 : System under study	Describe the Physical Twin system being studied. Include the type of system, its main components, operational domain, and key characteristics that make it suitable for digital twinning.
C_6 : DT services	Detail the specific services that the Digital Twin provides to users and the physical system. Include service types (monitoring, optimization, prediction, control, visualization), target users, and service capabilities.
C_{11} : Tooling and enablers	Describe the specific tools, technologies, frameworks, and enablers used to implement the Digital Twin. Include software platforms, development tools, simulation engines, databases, and any specialized technologies with their roles and functionalities.
$\mathcal{CC}_2$: Orchestration Cluster	
C_7 : Twinning time-scale	Describe the temporal aspects of the Digital Twin including synchronization frequencies between physical and digital components, time scales for different services, latency requirements, and temporal granularity needs.
C_8 : Multiplicities	Describe the multiplicities and hierarchical structure including multiple digital twin instances, centralized vs decentralized architectures, coordination mechanisms between twins, and scope of responsibilities for each instance.
C_{12} : DT constellation	Describe the overall orchestration and coordination of the Digital Twin system including architecture patterns, component integration, resource management, and system-wide coordination mechanisms.
C_{18} : Horizontal integration	Describe integration with external systems including data exchange protocols, interoperability standards, dependencies on external systems, and integration patterns with other Digital Twins or enterprise systems.
$\mathcal{CC}_3$: Components Cluster	
C_{10} : DT models and data	Describe the computational models and data components including model types (geometric, behavioral, analytical), data sources, data management approaches, model relationships, and their specific roles in the Digital Twin constellation.
C_2 : Physical acting components	Describe the physical actuators and control mechanisms including types of actuators, control capabilities, remote control interfaces, actuation ranges and limitations, safety constraints, and addressing mechanisms.
C_3 : Physical sensing components	Describe the sensing infrastructure including sensor types, measurement capabilities, spatial distribution, sampling frequencies, data transmission rates, accuracy and precision specifications, and data collection mechanisms.
C_{14} : Fidelity and validity	Describe model fidelity requirements, validation and verification methods, uncertainty sources and quantification, error detection and correction mechanisms, and quality assurance processes for models and data.
$\mathcal{CC}_4$: Connectivity Cluster	

Continued on next page...

Table 7: DTDF characteristic extraction prompts (continued).

<i>Continued from previous page</i>	
C_4 : Physical-to-virtual interaction	Describe data flows from physical to digital including data types transmitted, communication protocols, transmission frequencies, event triggers, data quality assurance, preprocessing steps, and data integration mechanisms.
C_5 : Virtual-to-physical interaction	Describe control flows from digital to physical including command types, control message formats, validation and authentication mechanisms, feedback systems, conflict resolution, and command execution confirmation.
C_{15} : DT technical connection	Describe the technical network infrastructure including communication protocols, network topology and architecture, bandwidth and latency requirements, reliability and redundancy mechanisms, and security protocols for network communication.
C_{16} : DT hosting/deployment	Describe the deployment infrastructure including hosting platforms (cloud/edge/on-premise), computational and storage requirements, scalability mechanisms, deployment models supported, and availability/reliability assurance.
CC_5: Life-cycle Cluster	
C_9 : Life-cycle stages	Describes the lifecycle phases in which the DT takes place. It also informs which representation phase the DT covers of its physical counterpart, i.e., as designed (ideal), as manufactured, or as operated.
C_{13} : Twinning process	Describe the engineering and development processes including methodologies used, quality assurance practices, testing and validation approaches, version control, and evolution management strategies.
C_{17} : Insights and decision making	Describe the decision support capabilities including types of insights generated, analytics and processing capabilities, decision making processes, user interaction methods, and confidence measures for recommendations.
C_{20} : Standardization	Describe standards compliance including relevant industry standards, specifications followed, certification requirements, compliance frameworks, and interoperability standards adopted.
CC_6: Governance Cluster	
C_{19} : Data ownership and privacy	Describe data governance including data ownership policies, privacy protection measures, compliance with data protection regulations, consent management, data anonymization, and data sharing agreements.
C_{21} : Security and safety	Describe security and safety measures including cybersecurity protocols, access control mechanisms, authentication systems, threat mitigation strategies, fail-safe mechanisms, risk assessment approaches, and safety-critical system considerations.

Table 8: RAG querying keywords for each characteristic cluster

Cluster	RAG Querying Keywords
$\mathcal{CC}_1$ (Purpose)	System under study: system purpose and objectives, twin description, operational context, performance objectives, stakeholder goals. DT services: monitoring and simulation, prediction and optimization, decision-support tools, control and actuation, visualization and interaction. DT enablers: technologies, tools, data acquisition pipeline, modeling and simulation tools, AI/ML components.
$\mathcal{CC}_2$ (Orchestration)	DT temporal aspects: synchronization frequencies, time scales, and temporal requirements. System multiplicities: multiple twins, distributed architectures, coordination mechanisms. DT constellation orchestration: system-wide coordination, component integration, service orchestration. External integration: data exchange with other systems, interoperability, horizontal integration patterns.
$\mathcal{CC}_3$ (Components)	DT models and data: computational models, data sources, model types, data management, model validation. Physical acting components: actuators, control mechanisms, remote control capabilities, actuation systems. Physical sensing components: sensors, measurement systems, data collection mechanisms, sensing technologies. Fidelity and validation: accuracy requirements, validation methods, uncertainty quantification, verification processes.
$\mathcal{CC}_4$ (Connectivity)	Physical-to-digital flows: sensor data transmission, communication protocols, data preprocessing, event handling. Digital-to-physical flows: command transmission, control signals, feedback mechanisms, command validation. Network infrastructure: communication protocols, network topology, bandwidth requirements, connection reliability. Deployment infrastructure: hosting platforms, computational resources, cloud/edge deployment, scalability considerations.
$\mathcal{CC}_5$ (Life-cycle)	DT lifecycle phases: development stages, deployment phases, evolution processes, maintenance activities. Engineering processes: development methodologies, quality assurance, testing approaches, version control. Decision support: insight generation, analytics capabilities, decision making processes, user interactions. Standards and compliance: industry standards, certification requirements, compliance frameworks, interoperability standards.
$\mathcal{CC}_6$ (Governance)	Data governance: data ownership policies, privacy regulations, data protection measures, consent management. Security considerations: cybersecurity measures, access control, authentication, threat mitigation, security protocols. Safety requirements: safety-critical considerations, fail-safe mechanisms, risk assessment, hazard analysis. Compliance frameworks: regulatory compliance, data protection laws, industry regulations, audit requirements.